

INTELIGÊNCIA ARTIFICIAL

Perceptron

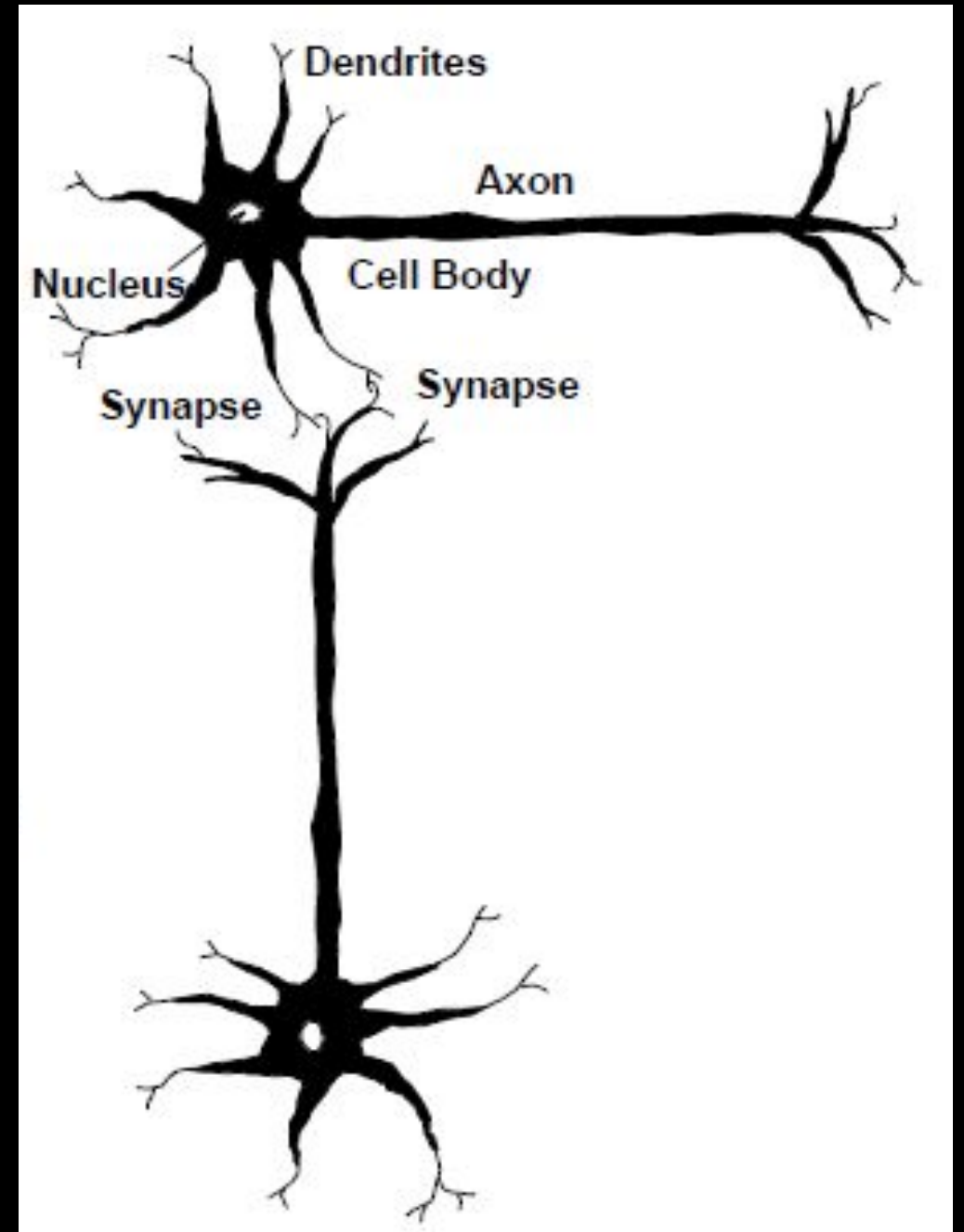
Karina Valdivia Delgado

Introdução

- Redes neurais artificiais são inspiradas na biologia e buscam reproduzir as **capacidades adaptativas** das redes no cérebro em diferentes tarefas.
- Um exemplo importante é o **Perceptron** e sua generalização, o Perceptron Multicamadas (Multilayer Perceptron - **MLP**)
- Suposição do Perceptron: **Os dados são linearmente separáveis.**

O neurônio biológico

- O neurônio é composto por um **corpo celular**, ramificações chamadas **dendritos** e um prolongamento denominado **axônio** que tem como função transmitir o sinal do corpo celular para suas extremidades.
- As extremidades dos axônios são conectadas com dendritos de outros neurônios pelas **sinapses**, formando grandes redes.



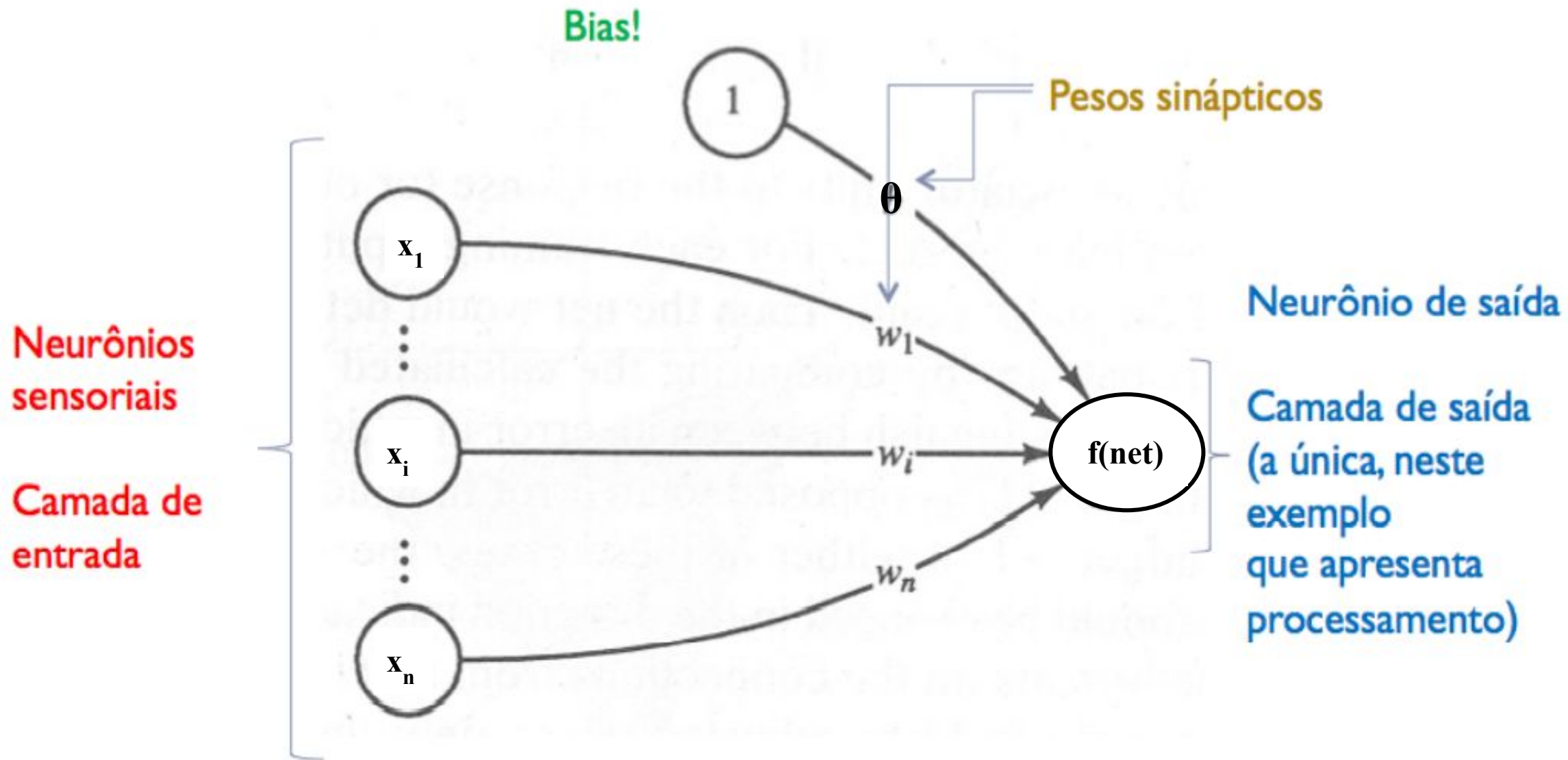
O neurônio biológico

- O aprendizado ocorre por sucessivas modificações nas sinapses que interconectam os neurônios.
- O **ajuste nas ligações** entre os neurônios durante o processo de aprendizado é uma das mais importantes características **das redes neurais artificiais**. A medida que novos eventos ocorrem, determinadas ligações entre neurônios são reforçadas, enquanto outras são enfraquecidas.

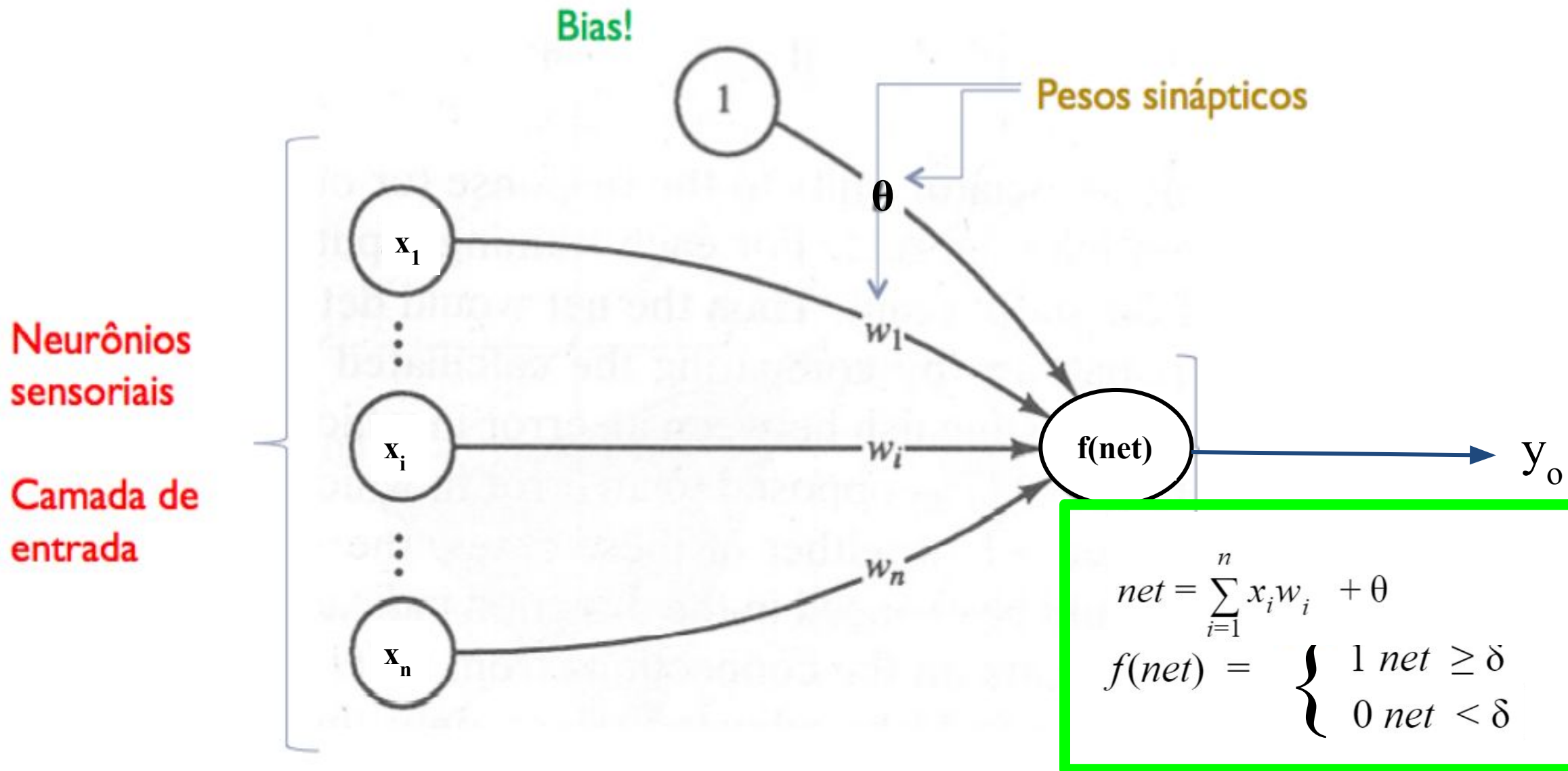
Primeiros modelos

- O primeiro modelo matemático do neurônio foi proposto por McCulloch & Pitts em 1943.
- Frank Rosenblatt em 1957 criou o modelo do **Perceptron**. Ele propôs um novo método para aprendizado supervisionado para reconhecimento de padrões.
- O Perceptron não é usado na prática, mas estudamos ele por razões históricas, é simples.

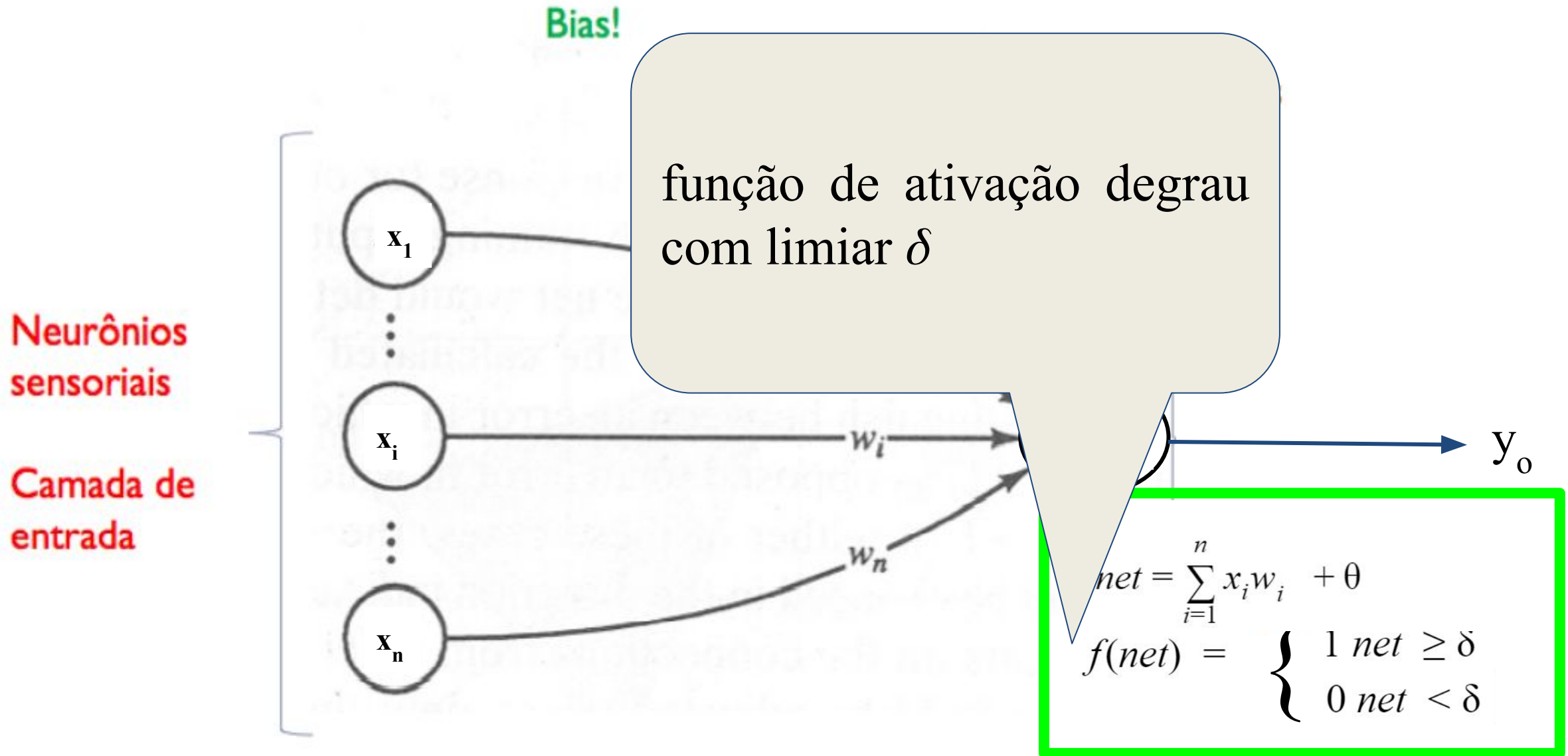
Perceptron



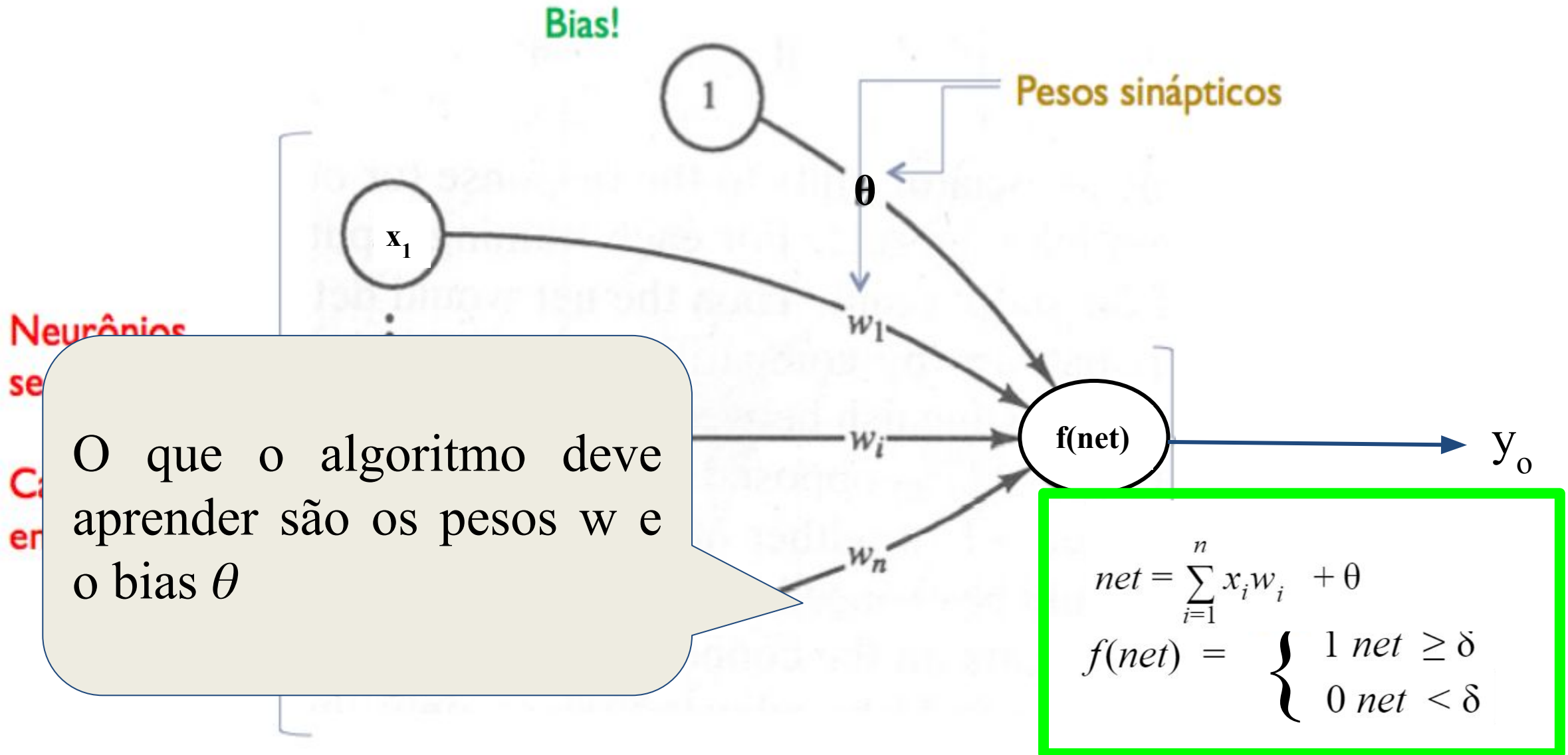
Perceptron



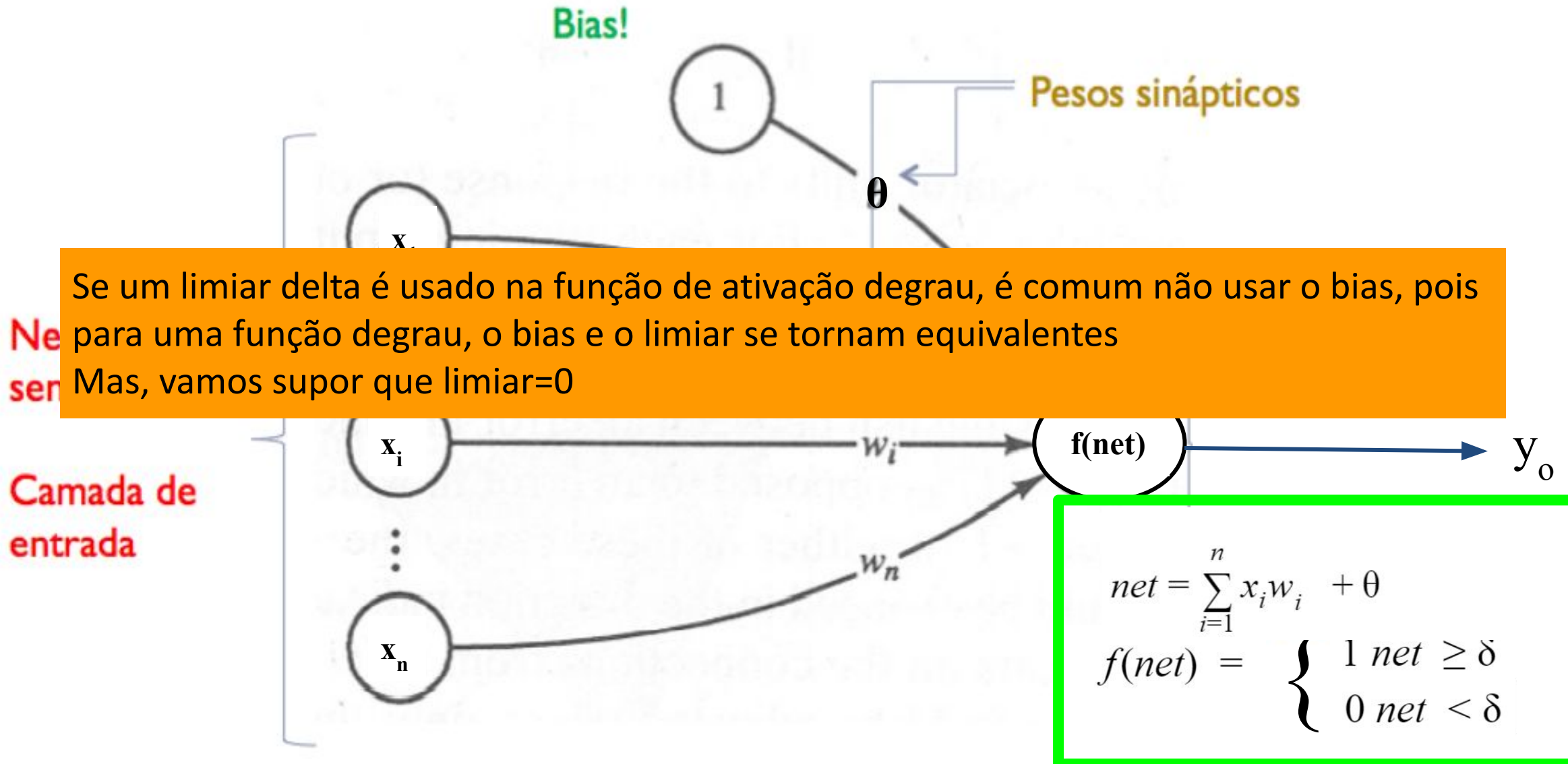
Perceptron



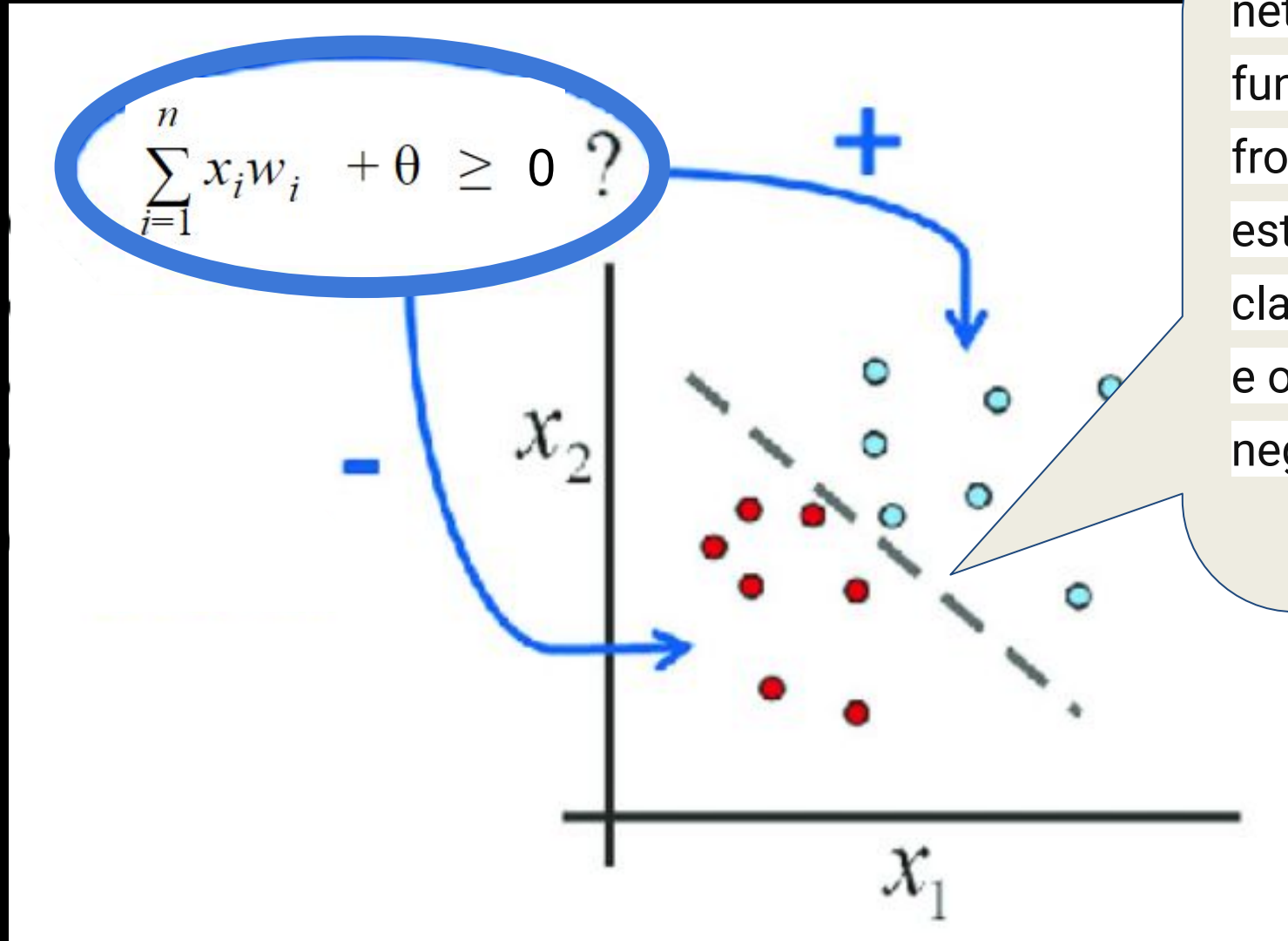
Perceptron



Perceptron

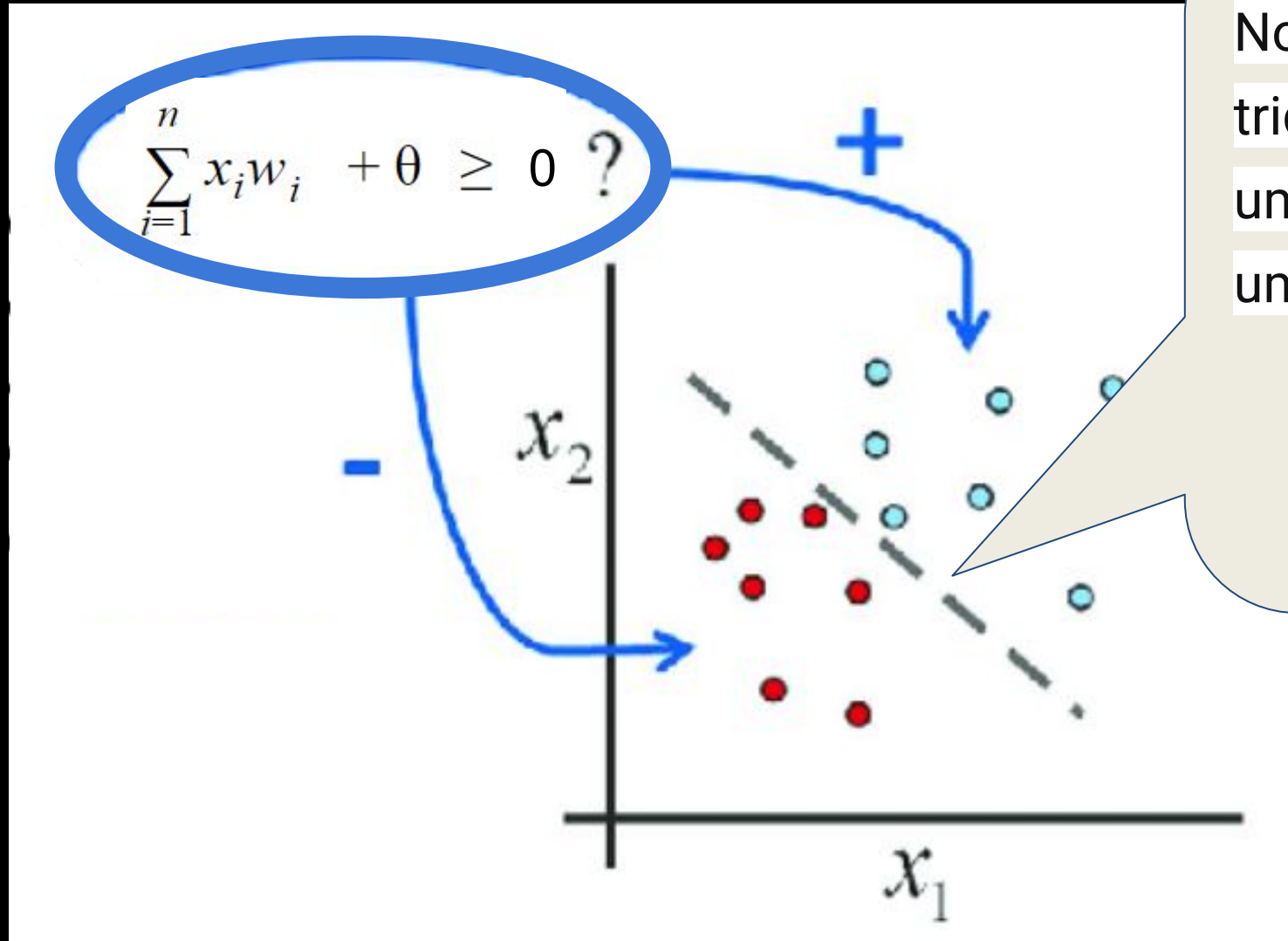


Perceptron



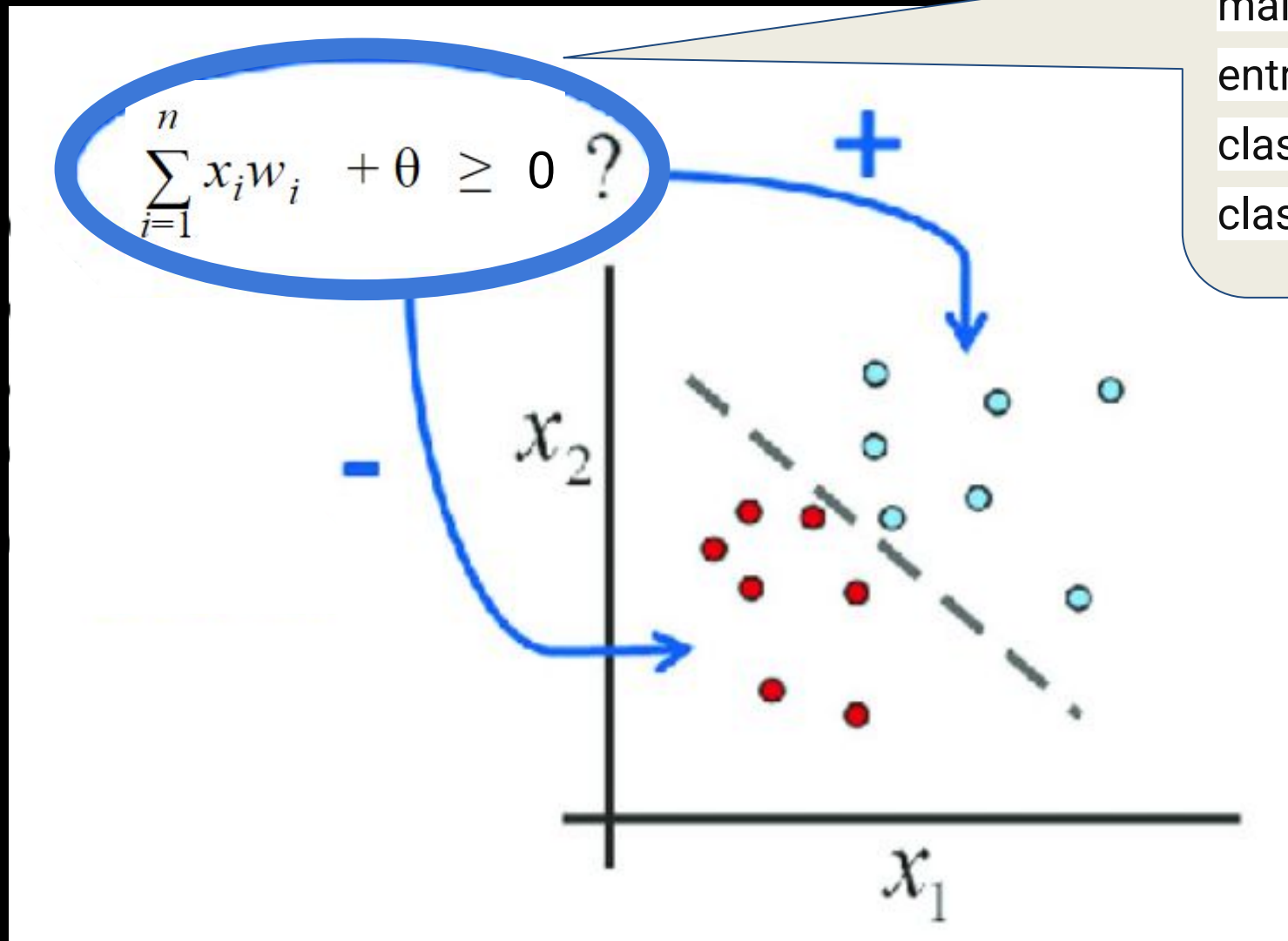
Se temos 2 características, $net=0$ define uma reta que funciona como uma fronteira de decisão, o que está acima desta reta será classificado como positivo e o que está abaixo como negativo.

Perceptron



No espaço tridimensional teremos um plano e em geral um hiperplano.

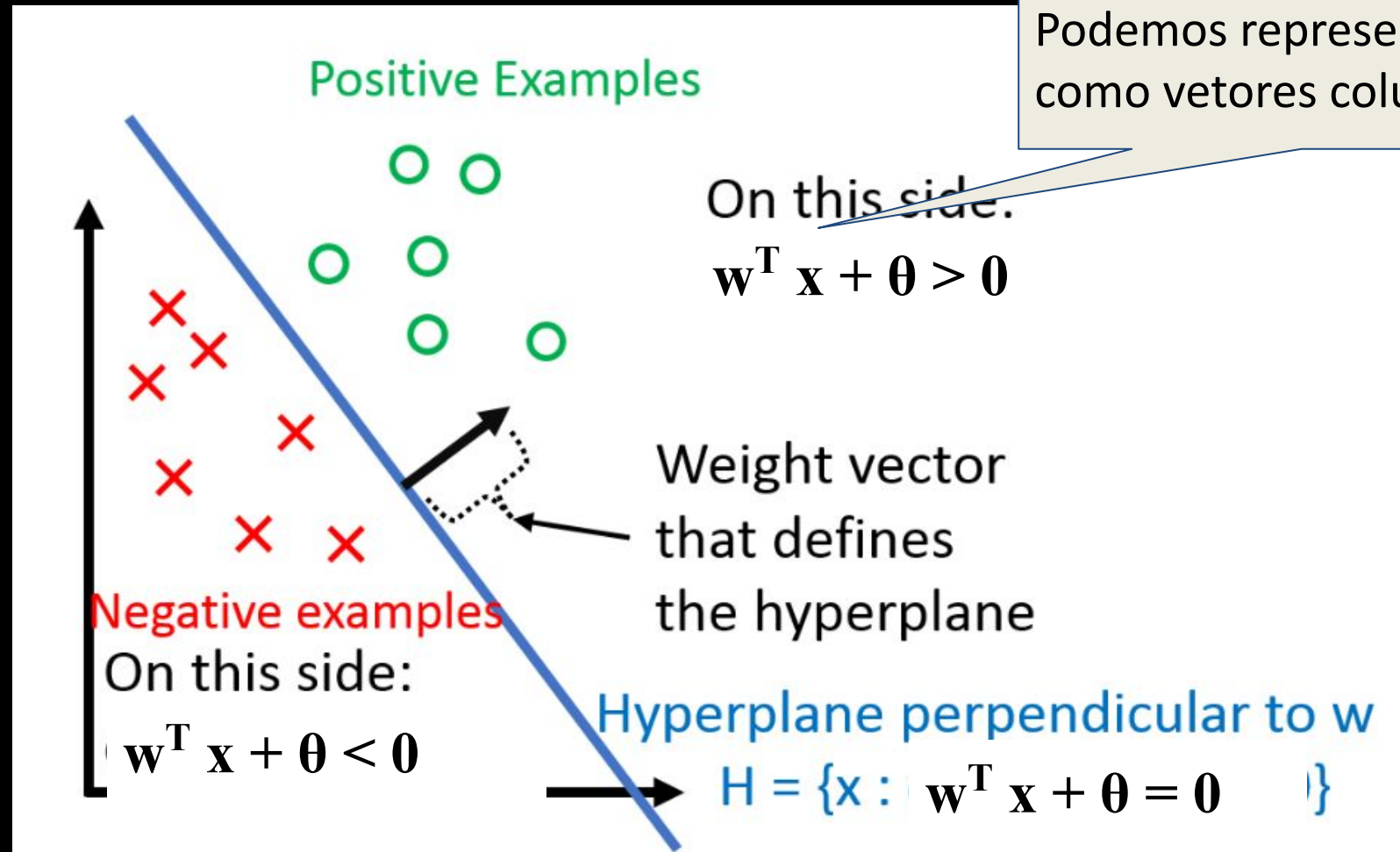
Perceptron



Se a soma ponderada for maior ou igual que delta, a entrada é atribuída a uma classe, caso contrário, à outra classe.

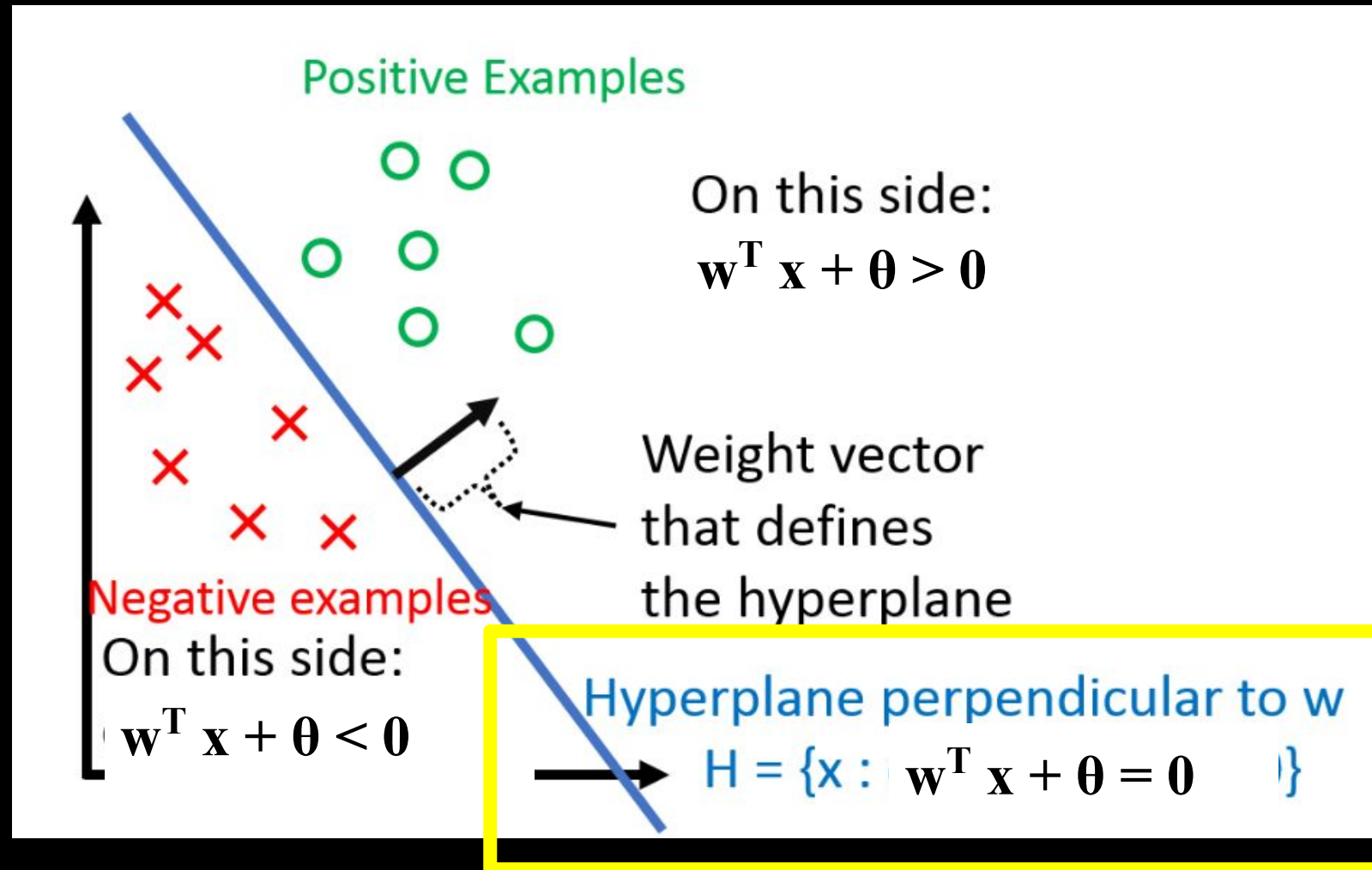
Perceptron

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



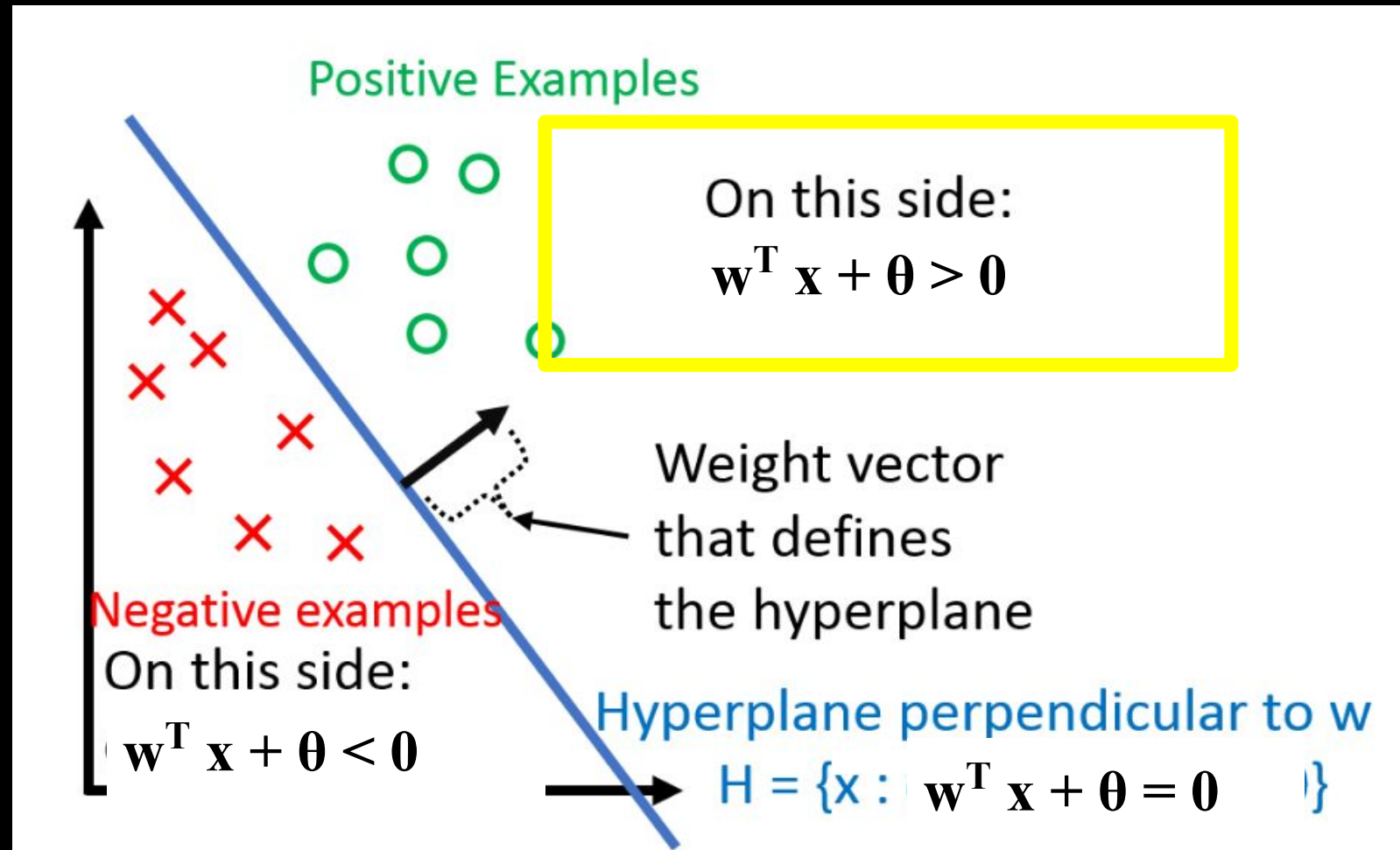
Perceptron

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



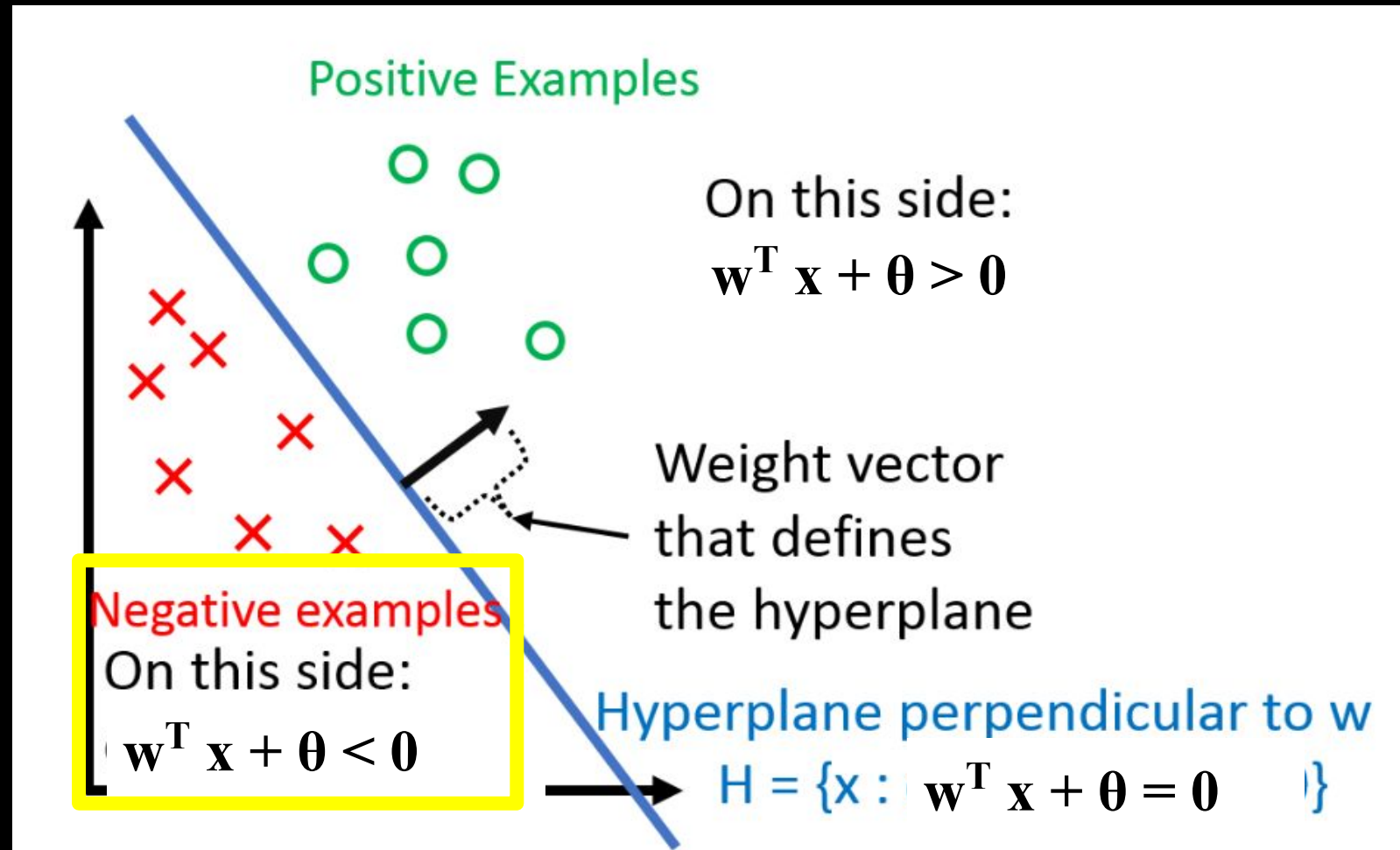
Perceptron

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Perceptron

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Perceptron: regra delta

Como sabemos os rótulos, podemos usar eles e compará-los com a saída do neurônio para calcular o erro

Perceptron: regra delta

O algoritmo do perceptron funciona da seguinte forma:

- Para cada exemplo (instância) de treinamento $\mathbf{x}$, a saída da rede y_o é calculada.
- Em seguida, é determinado o erro entre a saída desejada para este exemplo y e a saída da rede y_o , $\text{error} = y - y_o$.
- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

$$y_o = f(\text{net})$$

Perceptron: regra delta

O algoritmo do perceptron funciona da seguinte forma:

- Para cada exemplo (instância) de treinamento $\mathbf{x}$, a saída da rede y_o é calculada.
- Em seguida, é determinado o erro entre a saída desejada para este exemplo y e a saída da rede y_o , $\text{error} = y - y_o$.
- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$

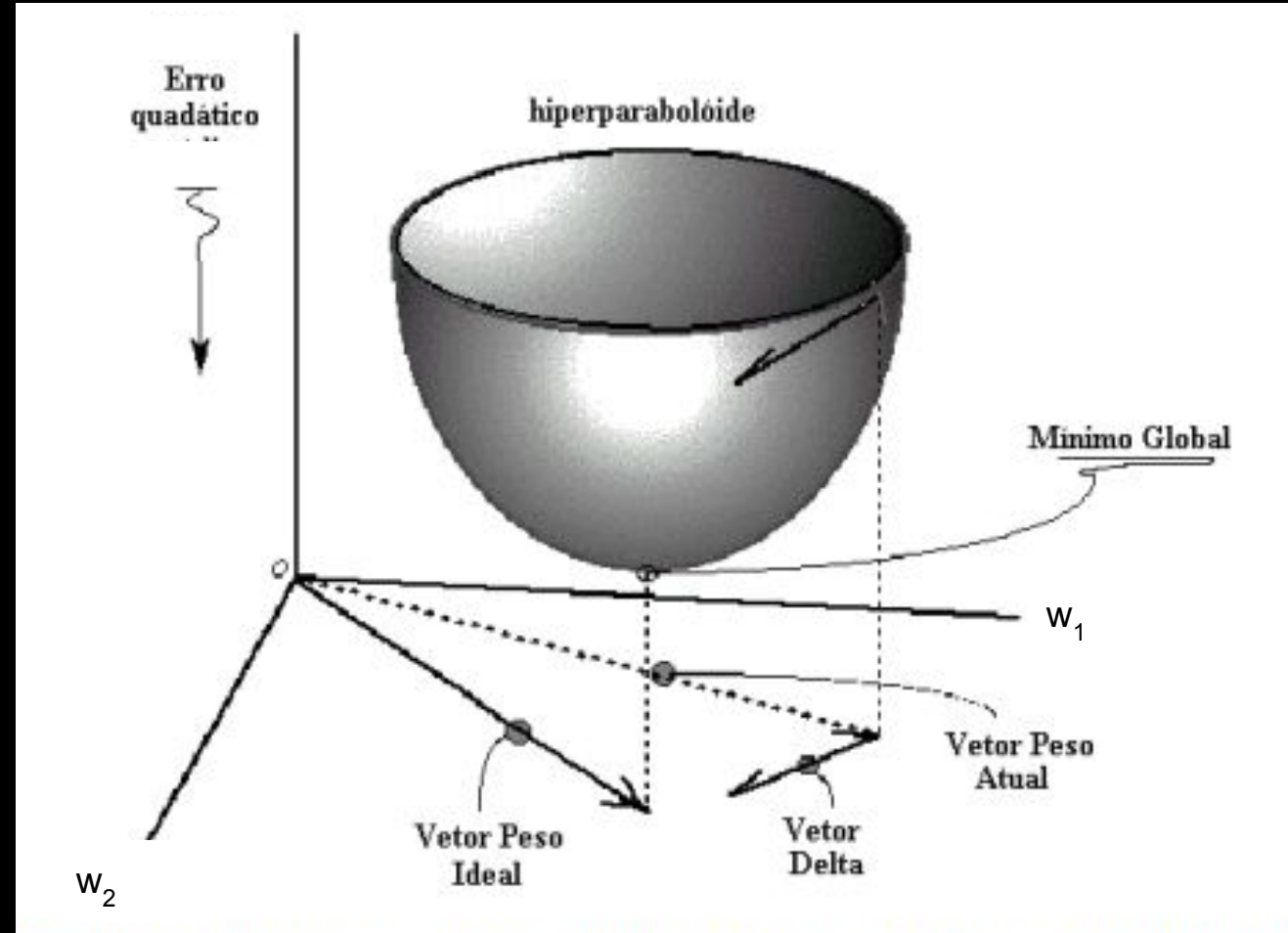
- o erro quadrático define um hiperparaboloide, uma função convexa
- Ela tem um mínimo global
- Desejamos minimizar esse erro (quadrático)

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

O algoritmo do perceptron funciona da seguinte forma:

- Para cada exemplo (instância) de treinamento x , a saída da rede y_o é calculada.
- Em seguida, é determinado o erro entre a saída desejada para este exemplo y e a saída da rede y_o , $\text{error} = y - y_o$.
- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$



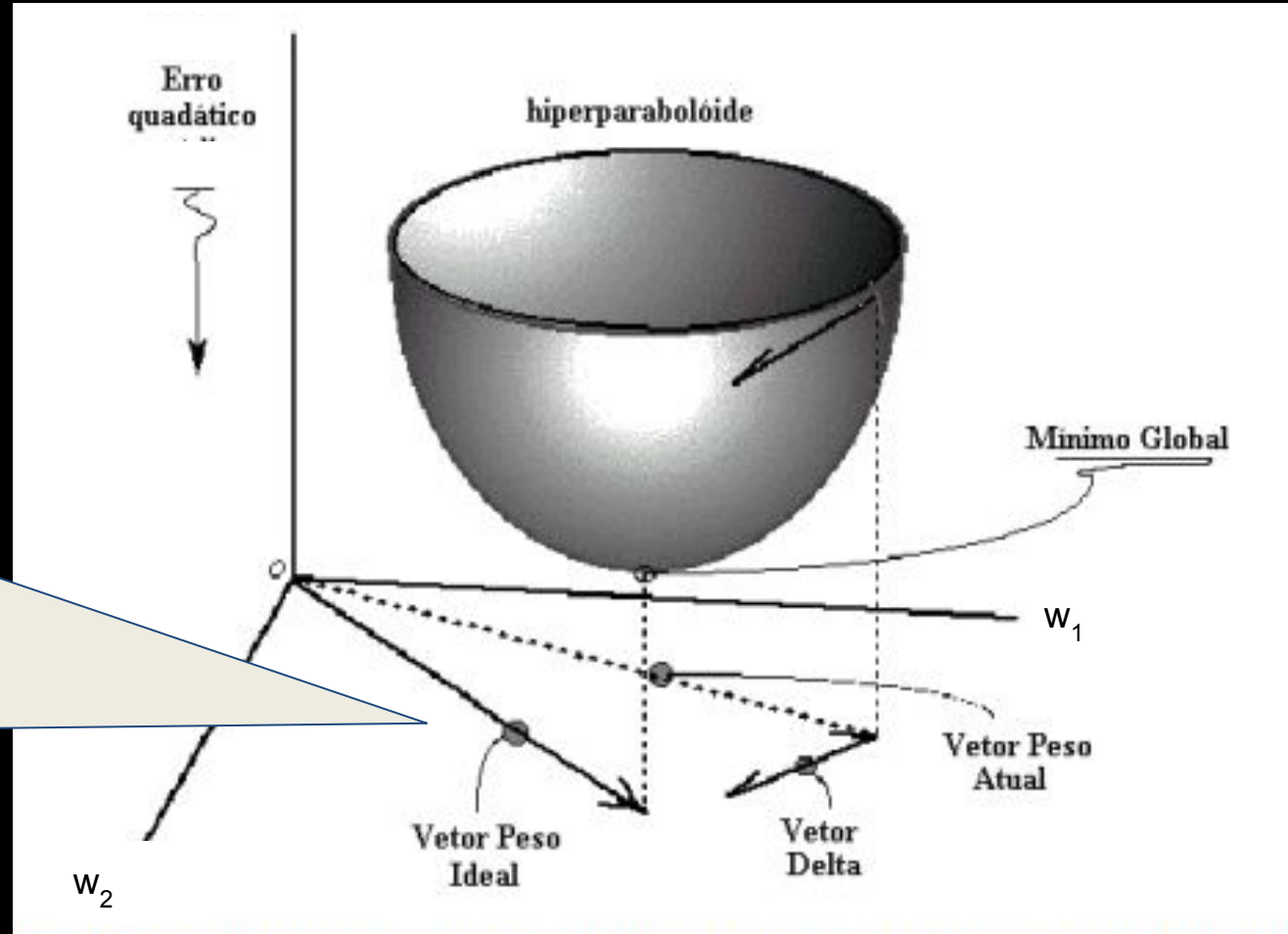
$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

O algoritmo do perceptron

função

- queremos encontrar os pesos associados ao mínimo global, ou seja
- desejamos encontrar esse vetor de peso ideal.



da rede y_0 , erro $y - y_0$.

- $\text{error}^2 = (y - y_0)^2$
 $= (y - f(\text{net}))^2$

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

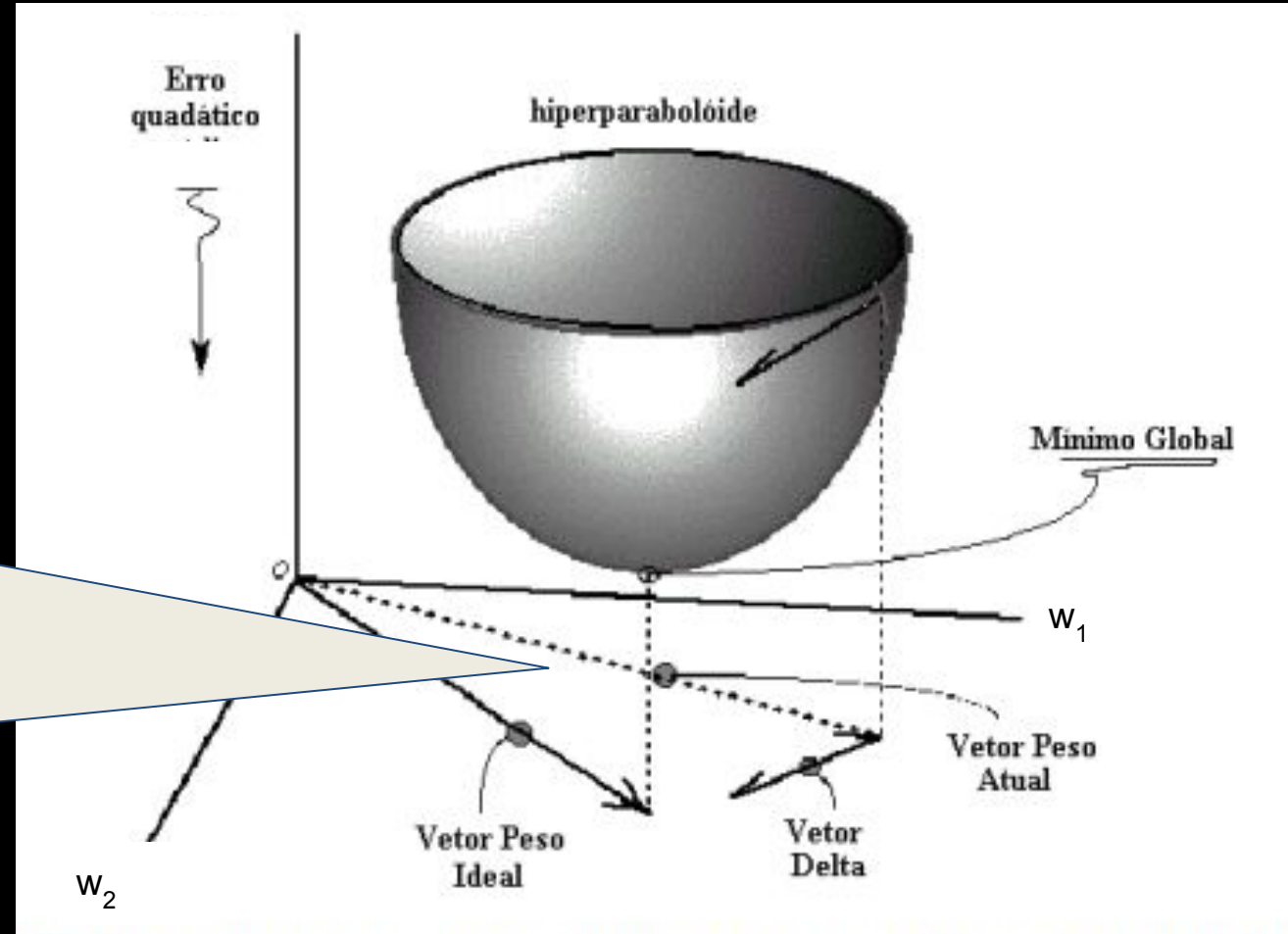
Perceptron: regra delta

O algoritmo do perceptron

função

- Vamos supor que meu vetor de peso atual é esse.

- $\text{error}^2 = (y - y_0)^2$
 $= (y - f(\text{net}))^2$



$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

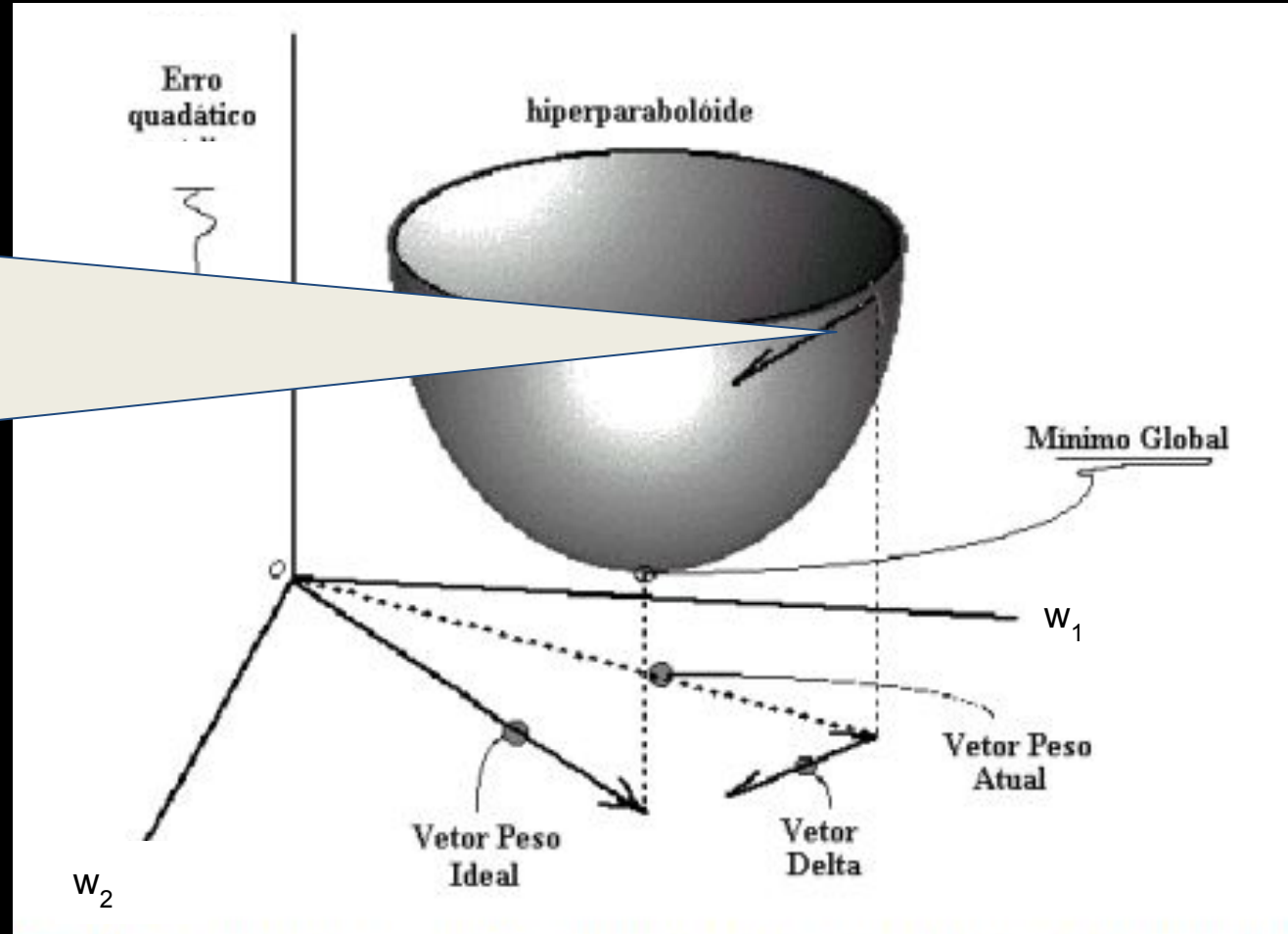
Perceptron: regra delta

O algoritmo do perceptron

função

- O erro quadrático para ele é esse valor
- Desejamos diminuir esse valor.
- Para isso vamos ter que modificar os pesos w_1 e w_2 .

o la la



- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

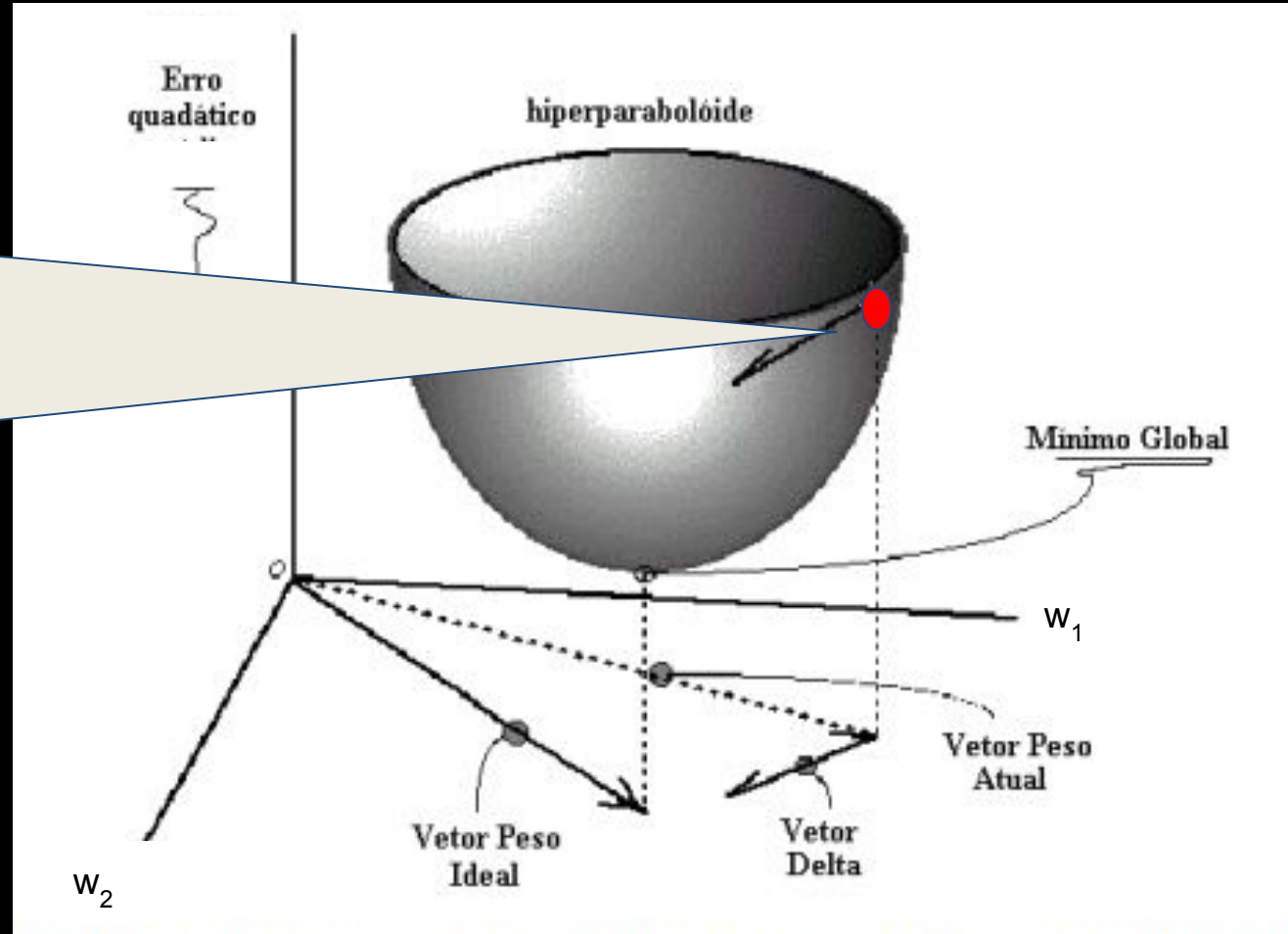
O algoritmo do perceptron

função

- Pensando em w_1

o la la

- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$



$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

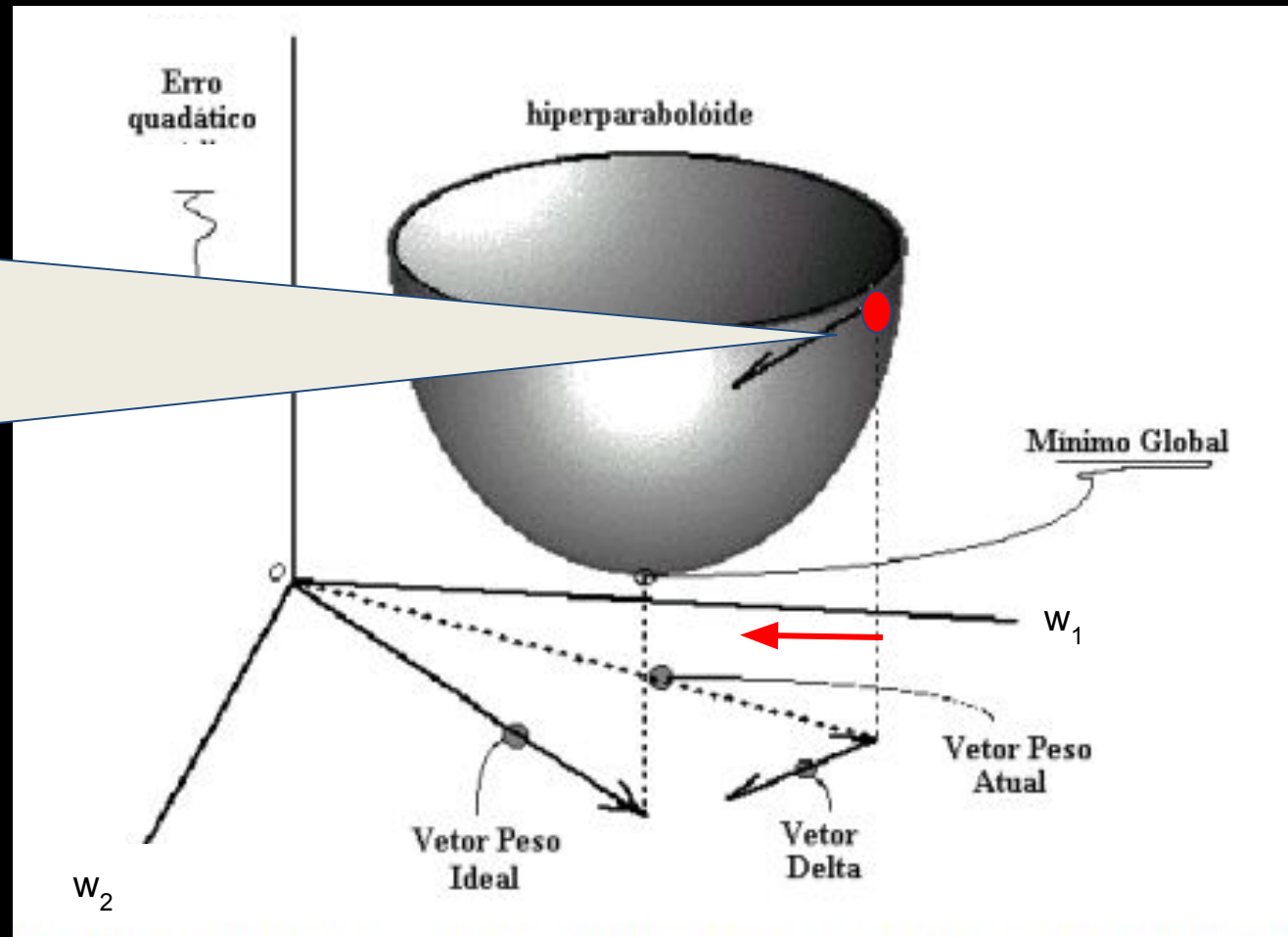
Perceptron: regra delta

O algoritmo do perceptron

- Pensando em w_1
- Se calcularmos a derivada com relação a w_1 , ela vai ser positiva

o
la
la

- $\text{error}^2 = (y - y_o)^2$
 $= (y - f(\text{net}))^2$



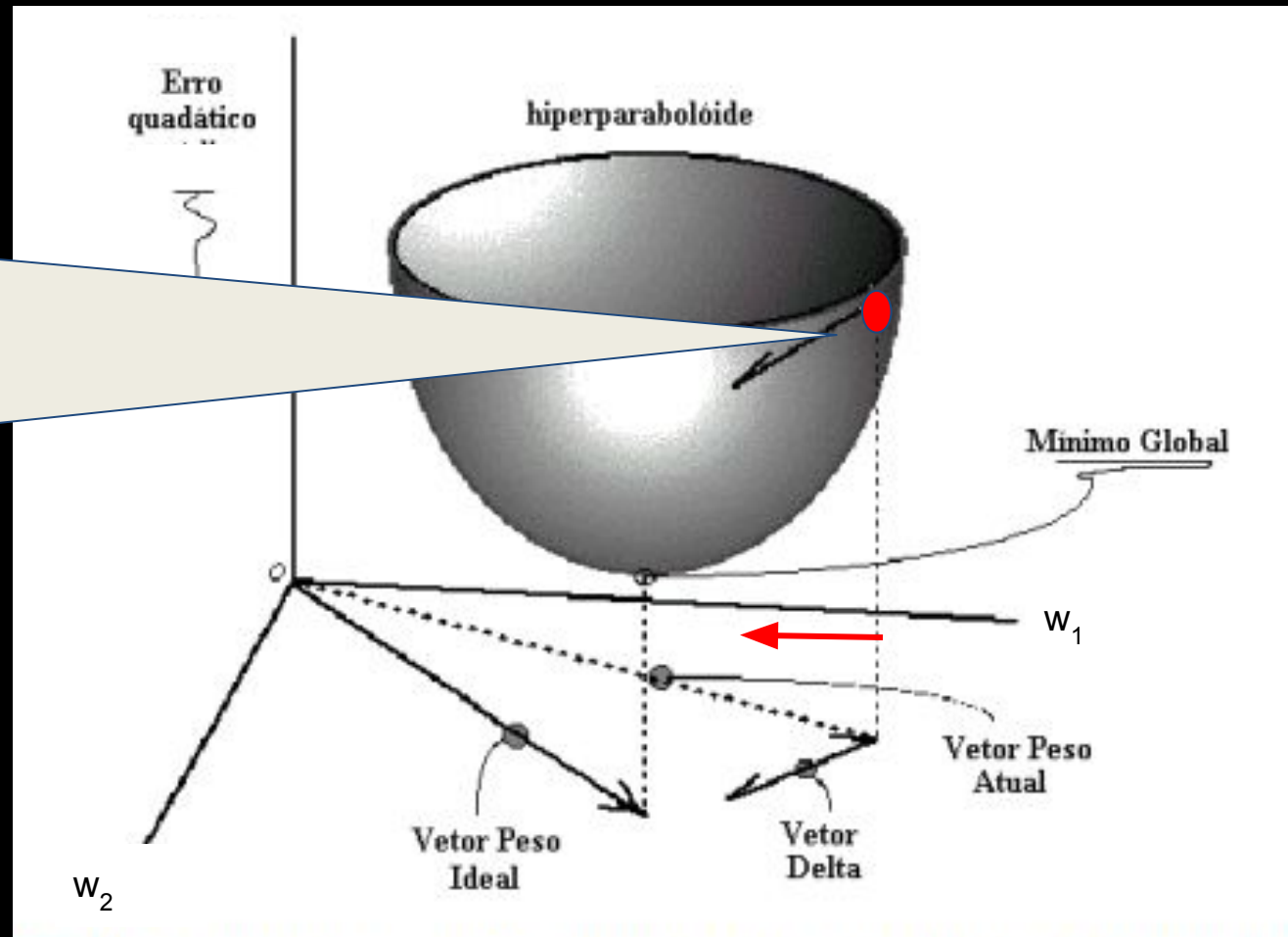
$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

O algoritmo do perceptron

- Pensando em w_1
- Se calcularmos a derivada com relação a w_1 , ela vai ser positiva
- Nesse caso eu sei que tenho que diminuir w_1

o
la
la



$$\text{error}^2 = (y - y_0)^2 \\ = (y - f(\text{net}))^2$$

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

O algoritmo do perceptron

fu

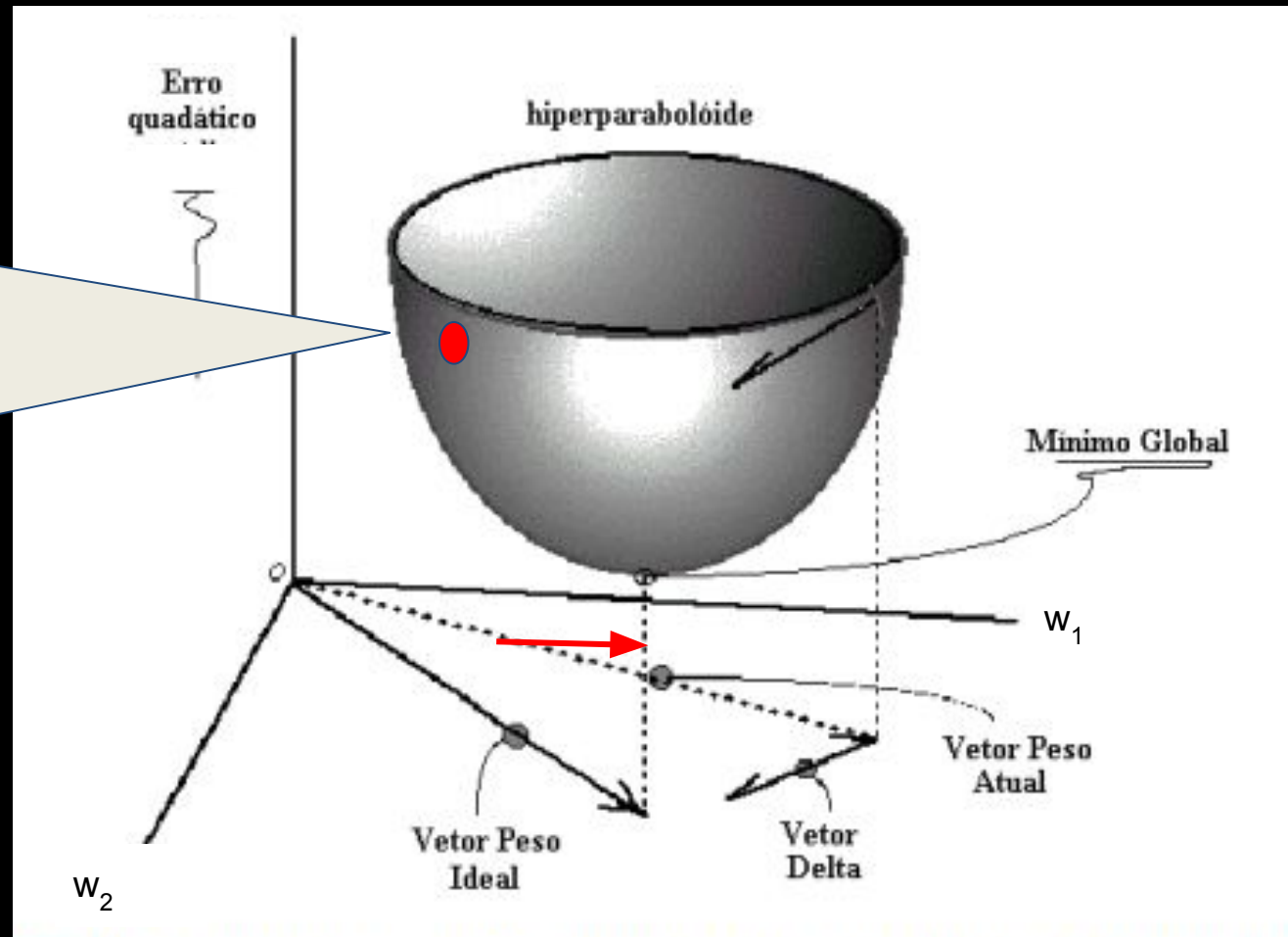
Se o meu erro quadrático é esse vermelho

- Pensando em w_1

- A derivada é negativa

- Nesse caso eu sei que tenho que aumentar w_1

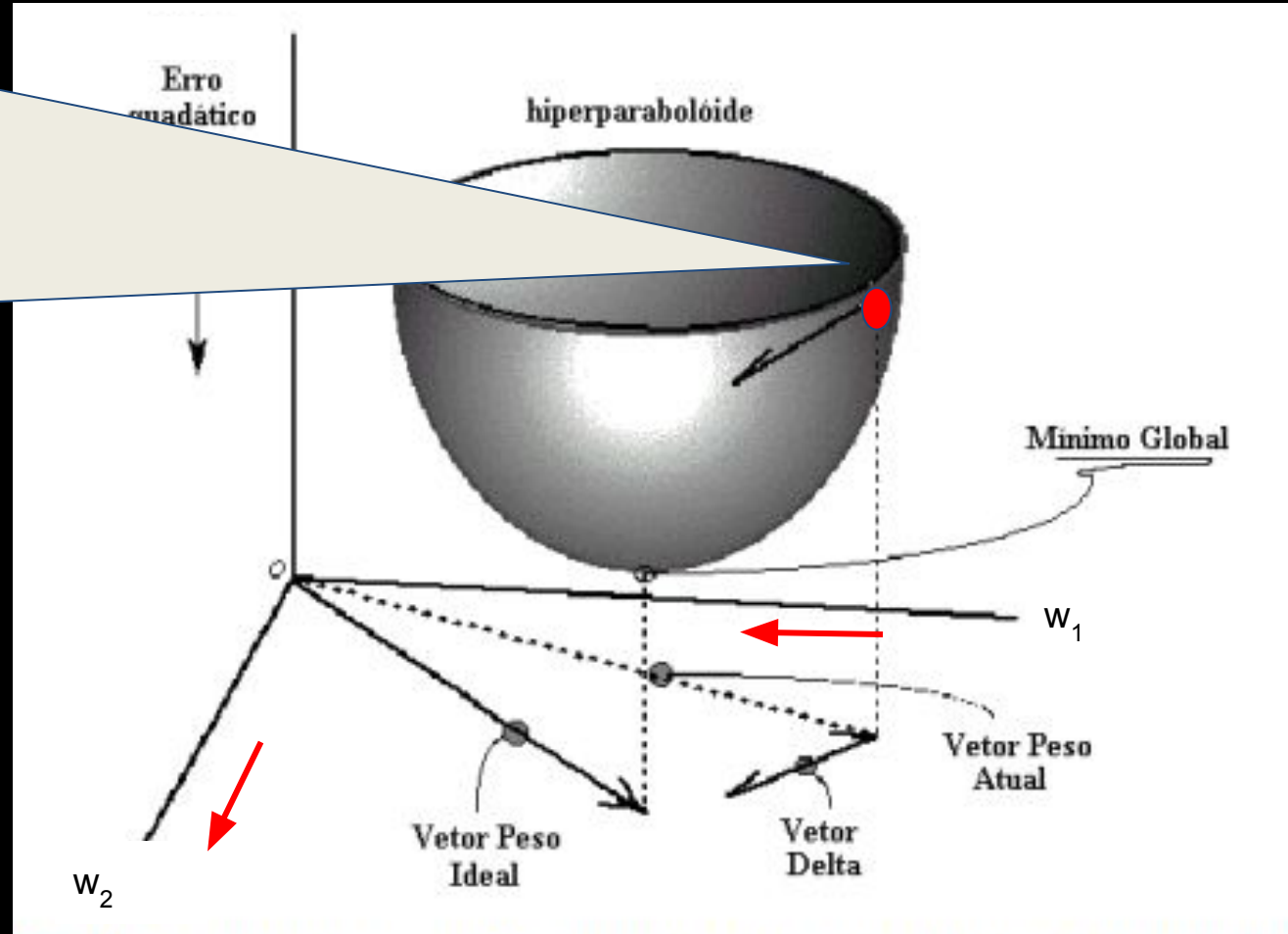
- $$\text{error}^2 = (y - y_o)^2$$
$$= (y - f(\text{net}))^2$$



$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

- O mesmo raciocínio pode ser feito para w_2 .

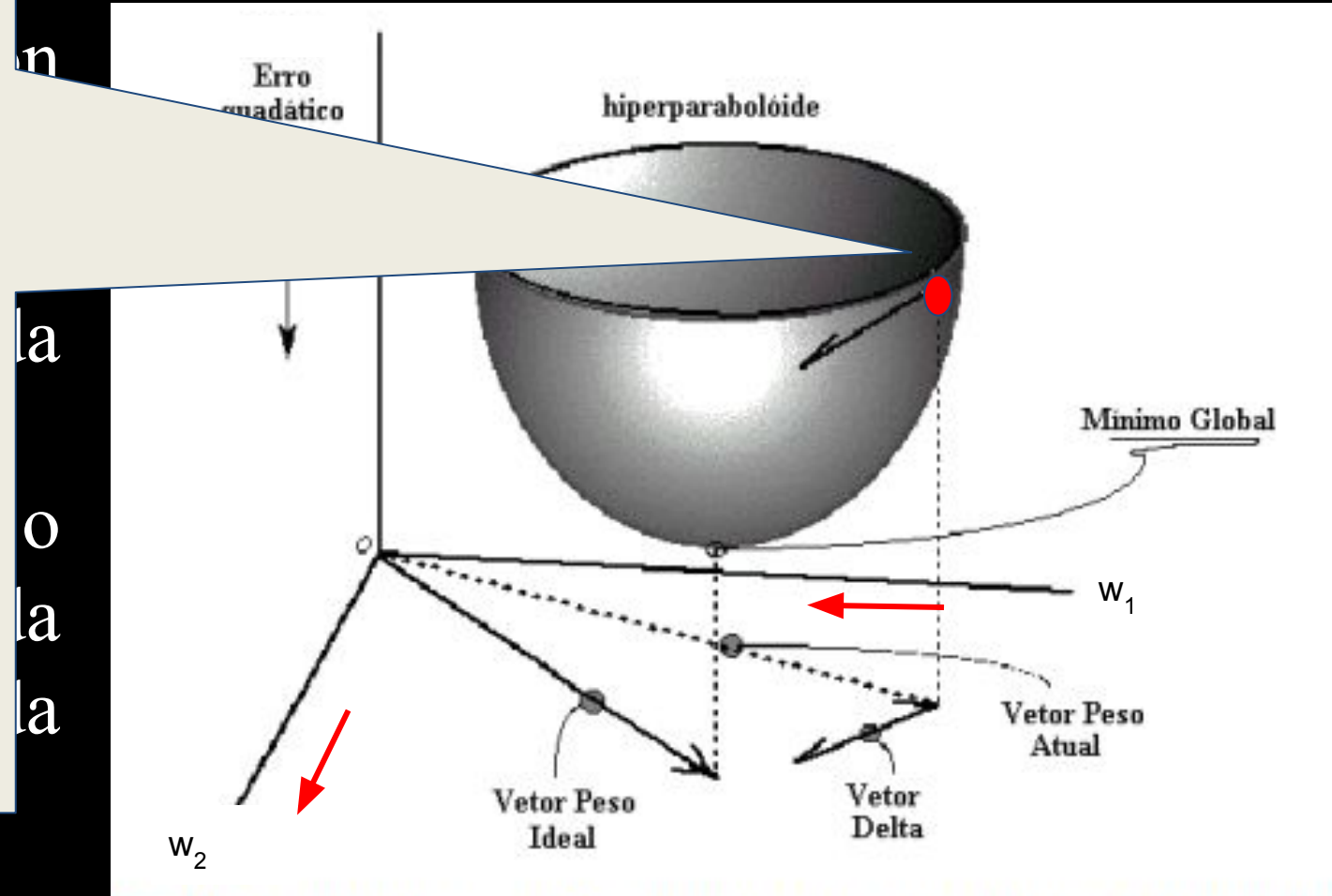


- $$\text{error}^2 = (y - y_o)^2$$
$$= (y - f(\text{net}))^2$$

$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

- Vou ter que diminuir um pouco w_1 e aumentar um pouco w_2
- Quanto é um pouco?
- Isso é controlado pela taxa de aprendizado η , também chamada de tamanho do passo



$$\text{error}^2 = (y - y_o)^2$$
$$= (y - f(\text{net}))^2$$

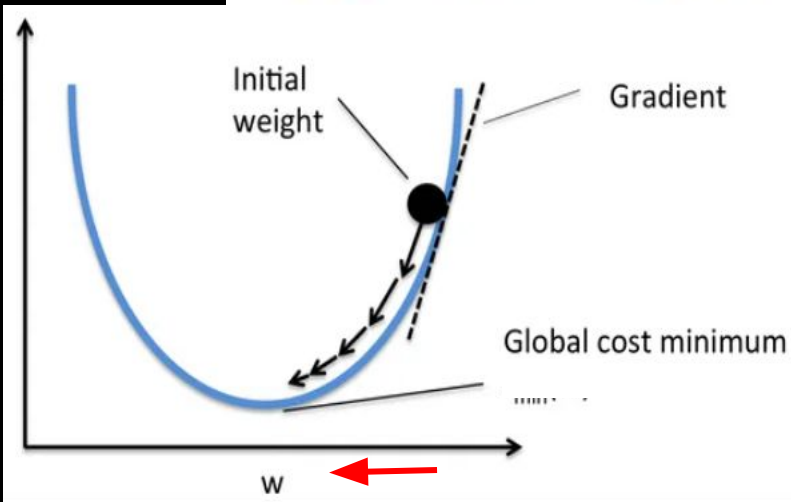
$$\text{net} = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta 2 * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta 2 * (y - y_0)(-1)$$



$$net = \sum_{i=1}^n x_i w_i + \theta$$

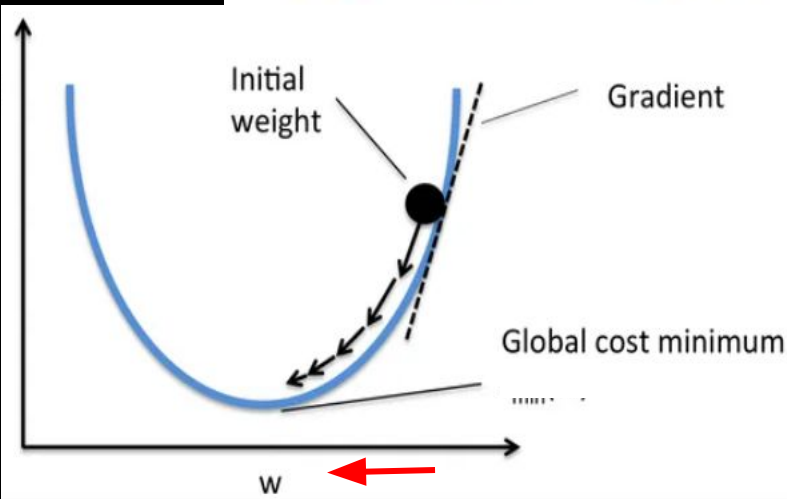
Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta 2 * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

- Se a derivada é positiva, eu vou ter que diminuir o valor de w , um pouco, é por isso que aqui é negativo.

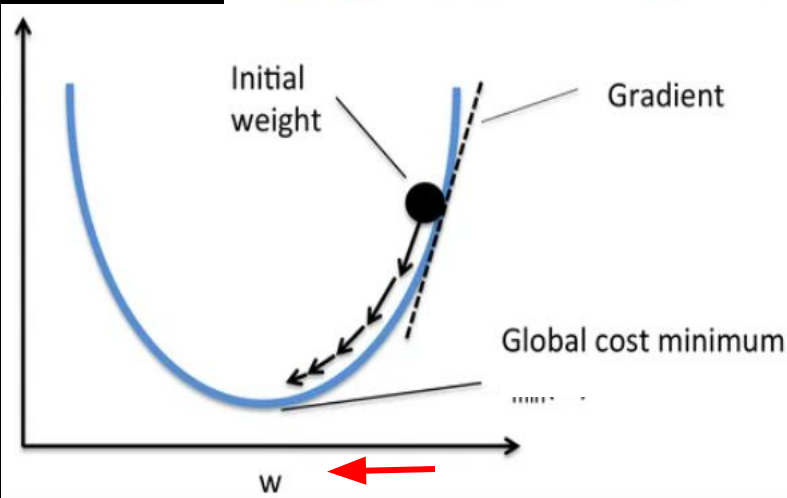


Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i}$$
$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta \Delta$$

- Usamos net diretamente e não $f(\text{net})$ para calcular a derivada.
- A função de ativação f de tipo de grau não é derivável

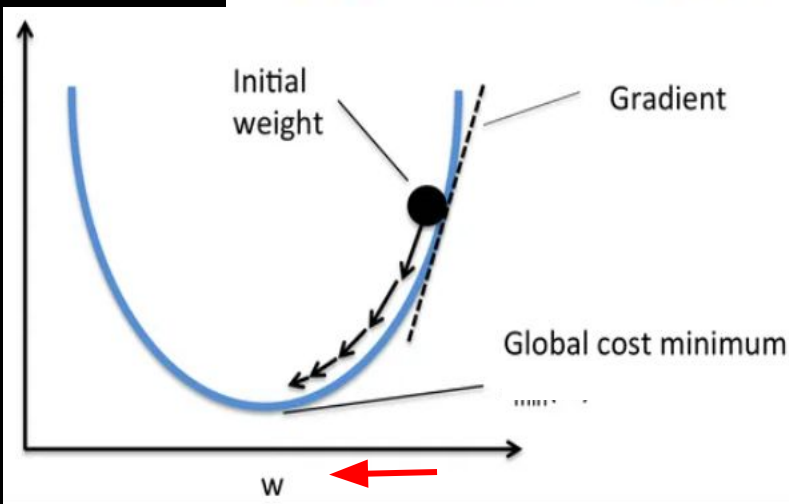


Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta 2 * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta 2 * (y - y_0)(-1)$$



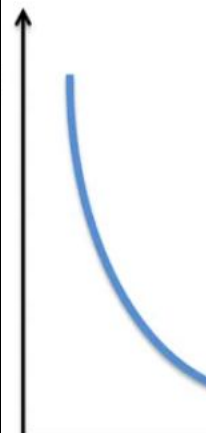
$$net = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta 2 * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta 2 * (y - y_0)(-1)$$



η é a taxa de aprendizado, geralmente é um valor entre 0 e 1, sem incluir o 0. Esse parâmetro controla o movimento dos pesos, i.e, permite realizar mudanças mais suaves

- não deixa haver uma mudança grande para um único exemplo
- se a mudança for mesmo necessária para que os pesos classifiquem corretamente um determinado exemplo, essa deve ocorrer gradualmente em várias “épocas”.

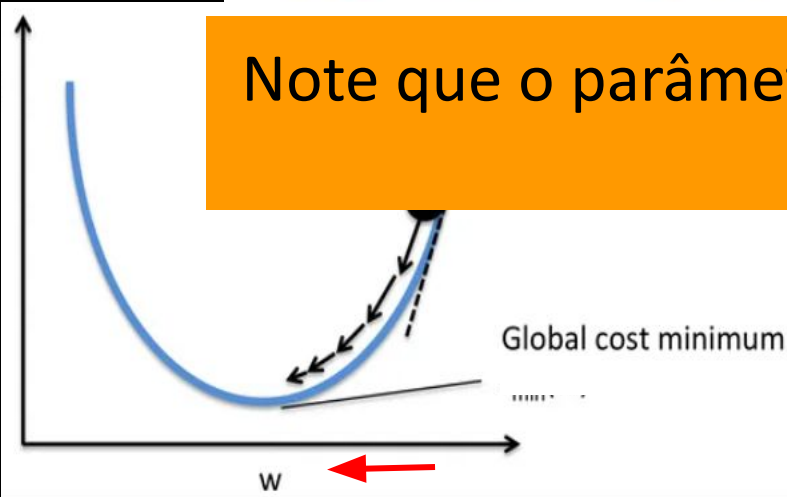
Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta 2 * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta 2 * (y - y_0)(-1)$$

Note que o parâmetro η pode incorporar a constante 2 do gradiente



$$net = \sum_{i=1}^n x_i w_i + \theta$$

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta * (y - y_0)(-x_i)$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta * (y - y_0)(-1)$$

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta * (y - y_0)(-x_i)$$
$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta * (y - y_0)(-1)$$

Intuitivamente o que fazemos é movimentar o peso em uma determinada direção para melhorar o desempenho da rede com relação ao erro². Essa direção está relacionada com a derivada parcial.

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta * (y - y_0)(-x_i)$$
$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta * (y - y_0)(-1)$$

O que é o aprendizado no Perceptron? é produzir adaptações dos w_i e θ usando essas equações em um processo iterativo.

treinamento (dataset, η , max_it, thresholdError)

initialize $\mathbf{w}$ θ // inicialize-os com 0 ou com números aleatórios [0, 1] ou [-1, 1]

$t = 1$

$sumError = \infty$

while ($t < \text{max_it}$ & $sumError > \text{thresholdError}$)

$sumError = 0$

for cada exemplo de entrada $\mathbf{x}, \mathbf{y}$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

$error = (y - y_0)$ // determinar o erro para o exemplo

$sumError = sumError + error^2$

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

$t=t+1$

end while

return $\mathbf{w}, \theta$

treinamento (dataset, η , max_it, thresholdError)

initialize $\mathbf{w}$ θ /

t = 1

sumError = ∞

while (t < max_it)

sumError = 0

for cada exemplo de entrada $\mathbf{x}$, $\mathbf{y}$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

$error = (y - y_0)$ // determinar o erro para o exemplo

$sumError = sumError + error^2$

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

t=t+1

end while

return $\mathbf{w}$, θ

dataset: conjunto de exemplos de treinamento

η : taxa de aprendizado

max_it: número máximo de iterações

thresholdError: erro máximo permitido

treinamento (dataset, η , max_it, thresholdError)

inicialize $\mathbf{w}$ e θ // inicialize-os com 0 ou com números aleatórios $[0, 1]$ ou $[-1, 1]$

$t = 1$

$sumError = \infty$

while ($t < \text{max_it}$)

$sumError =$

for cada ex

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

$error = (y - y_0)$ // determinar o erro para o exemplo

$sumError = sumError + error^2$

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

$t=t+1$

end while

return $\mathbf{w}$, θ

O treinamento é um processo iterativo por meio do qual os valores para os pesos são aprendidos a partir dos exemplos do meu dataset de treinamento.

Note que quando inicializamos $\mathbf{w}$ e θ , já foi criada uma reta, uma reta aleatória.

treinamento (dataset, η , max_it, thresholdError)

initialize $\mathbf{w}$ θ // inicialize-os com 0 ou com números aleatórios [0, 1] ou [-1, 1]

$t = 1$

$sumError = \infty$

while ($t < \text{max_it}$ & $sumError > \text{thresholdError}$)

época

$sumError = 0$

for cada exemplo de entrada $\mathbf{x}$, $\mathbf{y}$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

$error = (y - y_0)$ // determinar o erro para o exemplo

$sumError = sumError + error^2$

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

$t=t+1$

end while

return $\mathbf{w}$, θ

treinamento (dataset, η , max_it, thresholdError)

initialize $\mathbf{w}$ θ // inicialize-os com 0 ou com números aleatórios [0, 1] ou [-1, 1]

$t = 1$

$sumError = \infty$

while ($t < \text{max_it}$ & $sumError > \text{thresholdError}$)

$sumError = 0$

for cada exemplo de entrada $\mathbf{x}$, $\mathbf{y}$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

$error = (y - y_0)$ // determinar o erro para o exemplo

$sumError = sumError + error^2$

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

$t=t+1$

end while

return $\mathbf{w}$, θ

treinamento (dataset, η , max_it, thresholdError)

initialize $\mathbf{w}$ θ // inicialize-os com 0 ou com números aleatórios [0, 1] ou [-1, 1]

t = 1

sumError = ∞

while (t < max_it & *sumError* > thresholdError)

sumError = 0

for cada exemplo de entrada $\mathbf{x}$, $\mathbf{y}$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$y_0 = f(net)$ // calcular a resposta do neurônio de saída para o exemplo

error = $(y - y_0)$ // determinar o erro para o exemplo

sumError = *sumError* + *error*²

$w_1 = w_1 - \eta * (error)(-x_1)$ // atualizar os pesos

$w_2 = w_2 - \eta * (error)(-x_2)$

...

$w_n = w_n - \eta * (error)(-x_n)$

$\theta = \theta - \eta * (error)(-1)$ //atualizar o bias

end for

t=t+1

end while

return $\mathbf{w}$, θ

Note que o processo de aprendizado não para necessariamente depois de todos os exemplos de treinamento terem sido apresentados, veja que o for interno itera sobre todos os exemplos do conjunto de treinamento.

```
teste(x,w,  $\theta$  )
```

$$net = \sum_{i=1}^n x_i w_i + \theta$$

```
 $y_0 = f(net)$     // calcular a resposta do neurônio de saída para o exemplo
```

```
return  $y_0$ 
```

Agora que tenho o modelo (w e θ) que eu já calculei, já tenho a reta ou hiperplano.

Agora desejamos classificar um exemplo novo x

Funções de ativação

Função step (degrau) binária

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Função step (degrau) binária com limiar

$$f(x) = \begin{cases} 1 & \text{if } x \geq \delta \\ 0 & \text{if } x < \delta \end{cases}$$

Funções de ativação

Função step (degrau) bipolar

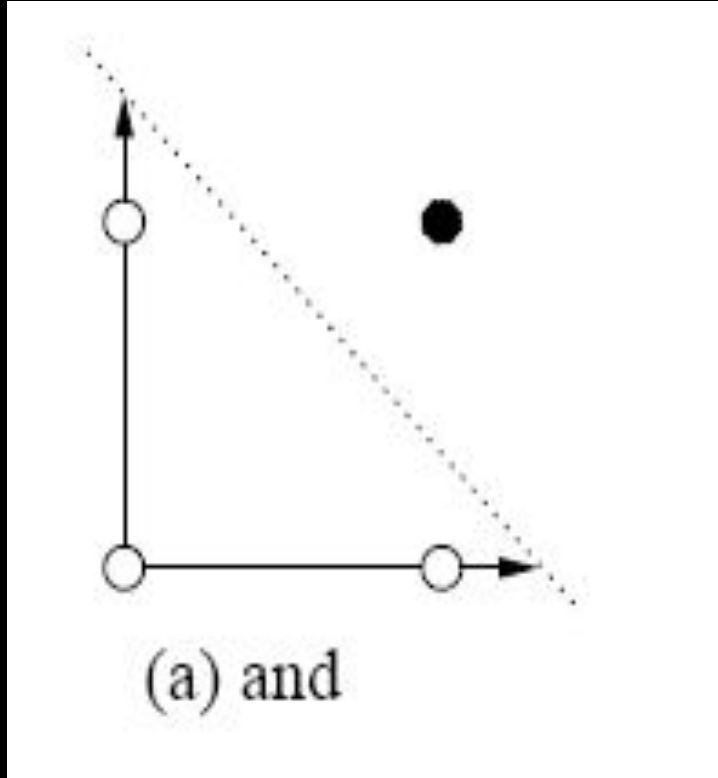
$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Função step (degrau) bipolar com limiar

$$f(x) = \begin{cases} 1 & \text{if } x \geq \delta \\ -1 & \text{if } x < \delta \end{cases}$$

Perceptron

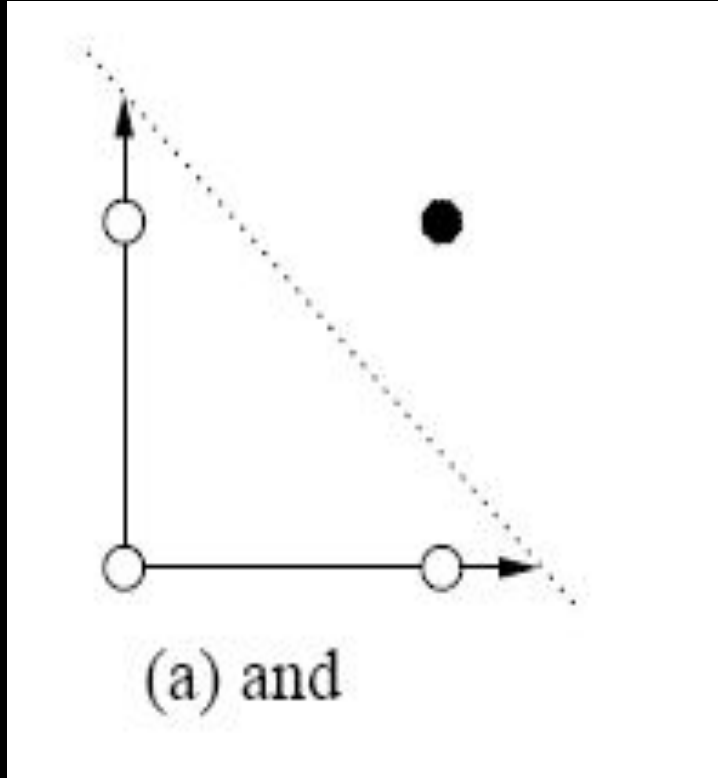
- Exemplo da comporta lógica AND



x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron

- Exemplo da comporta lógica AND

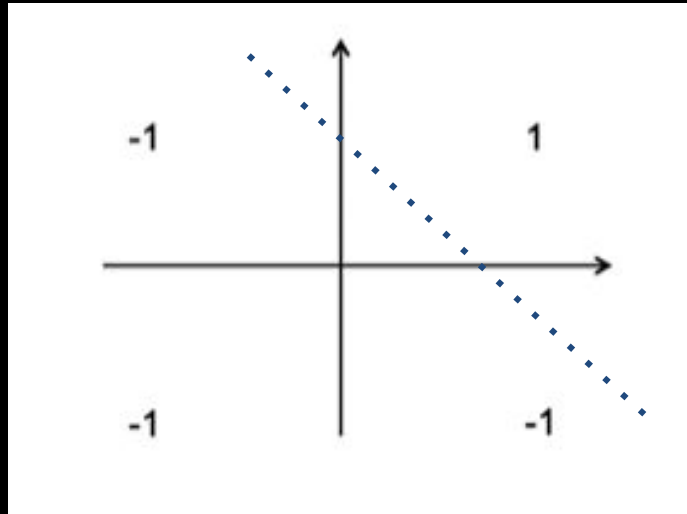


A porta lógica AND poderia ser a abstração de um problema de classificação. Por exemplo, vamos supor que temos 2 características de um produto, se o produto têm as duas características positivas ele é um bom produto, caso contrário ele não é um bom produto.

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron

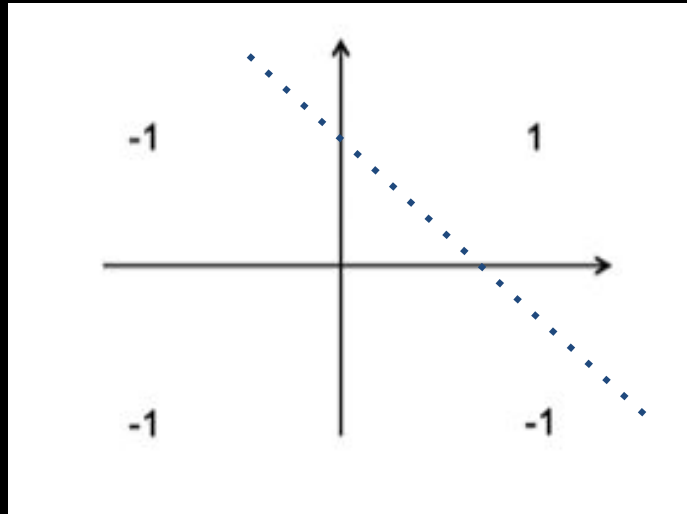
- Exemplo da comporta lógica AND



x1	x2	y
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Perceptron

- Exemplo da comporta lógica AND

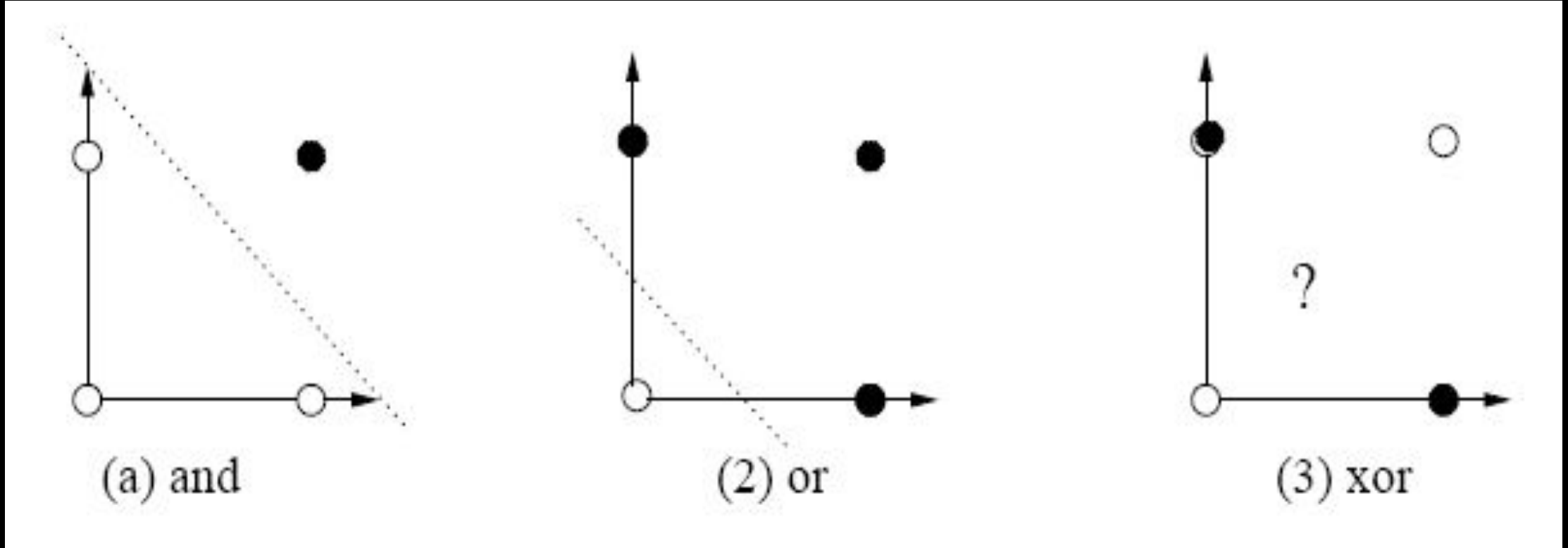


Se conhecemos os valores que x_1 e x_2 podem assumir e conhecemos os valores que y deveria assumir para cada um dos pares de valores de entrada, nosso problema é encontrar os valores de w_1 , w_2 e θ que permitam posicionar a reta de forma que as respostas desejadas para y sejam obtidas, considerando a função de ativação.

x_1	x_2	y
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Perceptron

- O perceptron só é capaz de aprender a solução de problemas linearmente separáveis.



Observações sobre o algoritmo de
treinamento usado pelo Perceptron
original

Perceptron: regra delta

- O vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t+1) = w_i(t) - \eta \frac{\partial (y - y_0)^2}{\partial w_i} = w_i(t) - \eta * (y - y_0)(-x_i)$$
$$\theta(t+1) = \theta(t) - \eta \frac{\partial (y - y_0)^2}{\partial \theta} = \theta(t) - \eta * (y - y_0)(-1)$$

Intuitivamente o que fazemos é movimentar o peso em uma determinada direção para melhorar o desempenho da rede com relação ao erro². Essa direção está relacionada com a derivada parcial.

Perceptron: regra simples

- Se $y_0 \neq y$, o vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$
$$\theta(t + 1) = \theta(t) + \eta * y$$

Intuitivamente o que fazemos é movimentar o peso em uma determinada direção para melhorar o desempenho da rede se o valor calculado pela rede é diferente do rótulo do meu exemplo de treinamento.

Perceptron: regra simples

- Se $y_0 \neq y$, o vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$
$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

$y_0 = 1$ (saída calculada pela rede) e $y = -1$ (rótulo do exemplo de treinamento) :

net está alto (está acima de 0 ou de δ), e **net precisa diminuir**.

Perceptron: regra simples

- Se $y_0 \neq y$, o vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$
$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

$y_0 = -1$ (saída calculada pela rede) e $y = 1$ (rótulo do exemplo de treinamento) :

net está baixo (está abaixo de 0 ou de δ), e **net precisa aumentar**.

Perceptron: regra simples

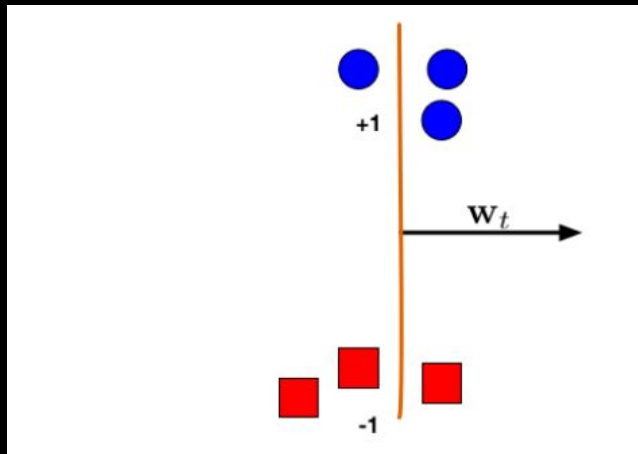
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



- Exemplos positivos azuis
- Exemplos negativos vermelhos
- net é um hiperplano que é perpendicular a w

Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

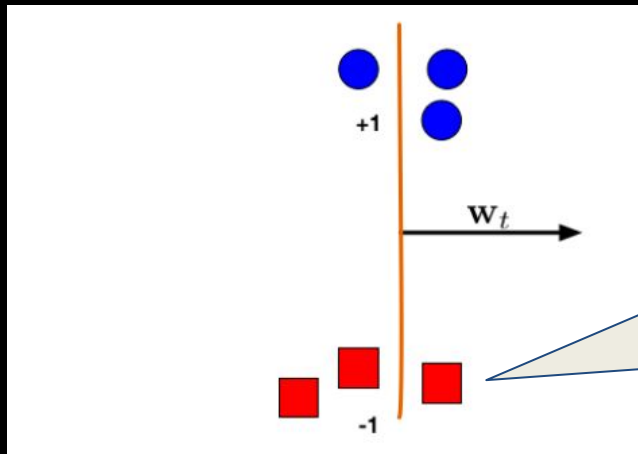
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



- Não está classificado corretamente
 $y_0 = 1$
 $y = -1$

Lecture 5

"Perceptron"

-Cornell CS4780
SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=wl7gVvI-HuY>

Perceptron: regra simples

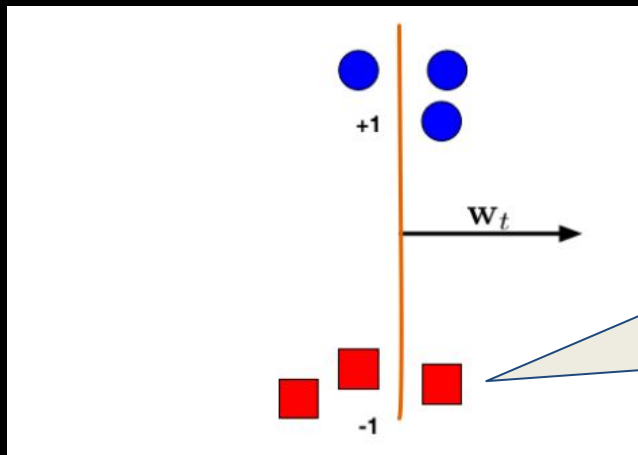
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



- Não está classificado corretamente
 $y_0 = 1$
 $y = -1$
- net precisa diminuir

Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

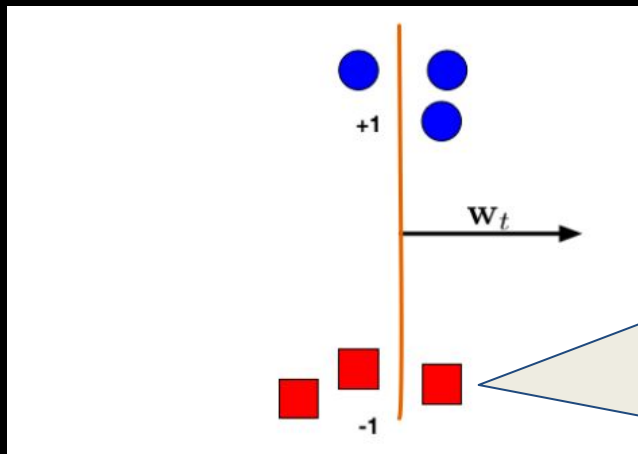
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



- Não está classificado corretamente
 $y_0 = 1$
 $y = -1$
- net precisa diminuir
- w precisa diminuir
- subtrair x de w

Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

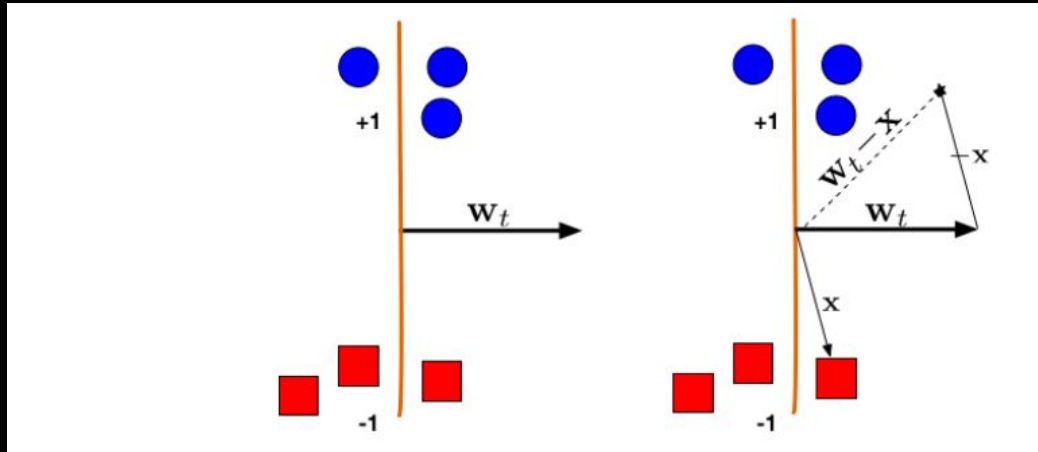
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

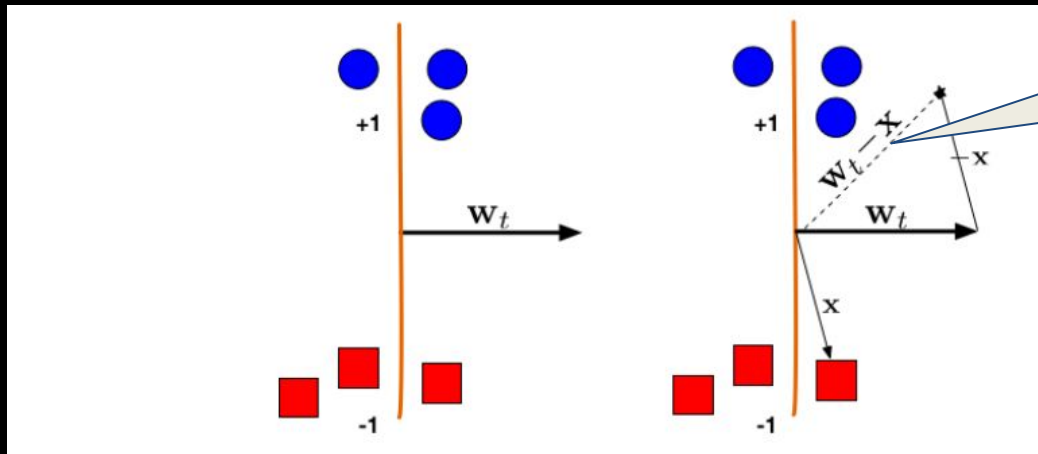
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Esse é meu
novo w

Lecture 5

Perceptron"

Cornell CS4780

P17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

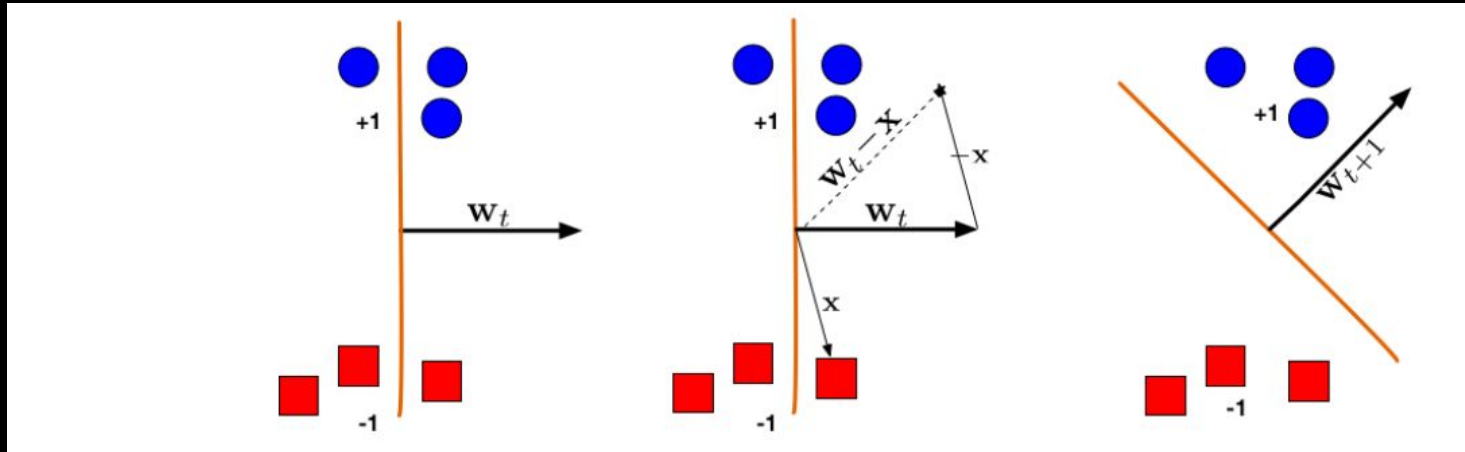
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

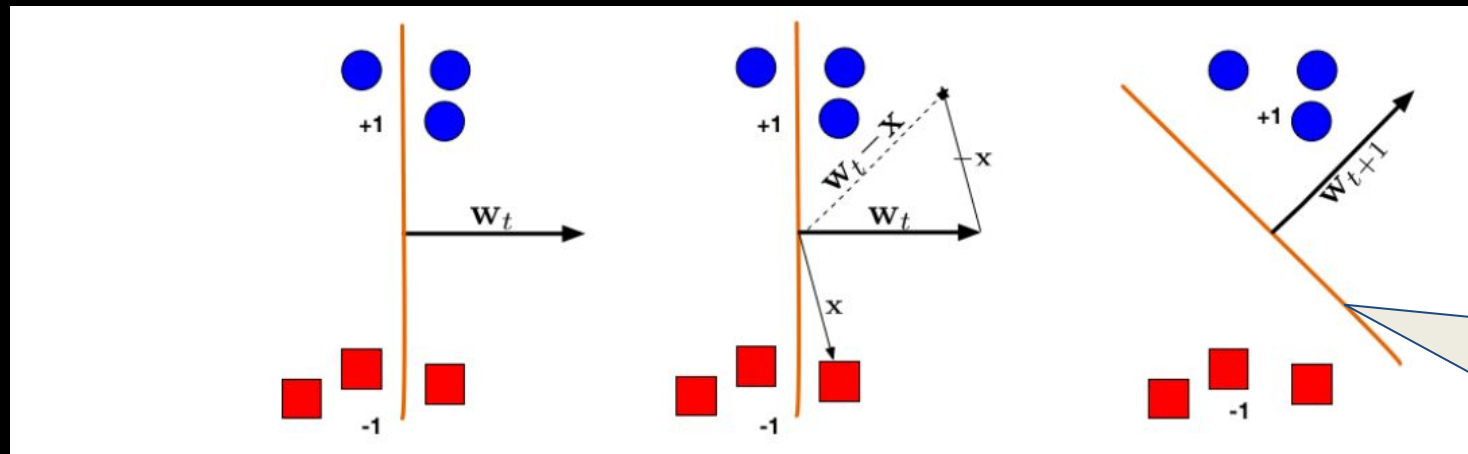
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

• net é um hiperplano
que é perpendicular a
w

Perceptron: regra simples

$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

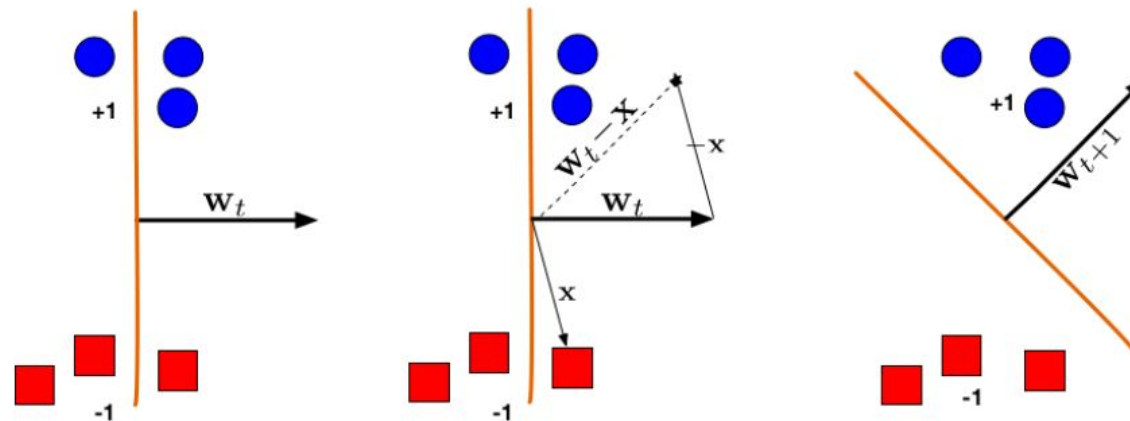
$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

Geometric Intuition

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



*Illustration of a Perceptron update. (Left:) The hyperplane defined by w_t misclassifies one red (-1) and one blue (+1) point. (Middle:) The red point x is chosen and used for an update. Because its label is -1 we need to **subtract** x from w_t . (Right:) The updated hyperplane $w_{t+1} = w_t - x$ separates the two classes and the Perceptron algorithm has converged.*

Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

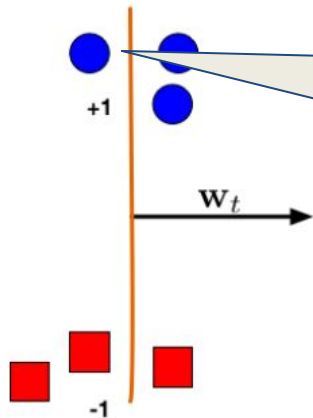
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture5>



- Não está classificado corretamente
 $y_0 = -1$
 $y = 1$
- net precisa ?
- w precisa ?
- qual operação devo aplicar a x e w?

Lecture 5
"Perceptron"
Cornell CS4780

Levinberger

até 24:30

[www.youtube.c](https://www.youtube.com/watch?v=w17gVvI)

[ach?v=w17gVvI](https://www.youtube.com/watch?v=w17gVvI)

edY

Perceptron: regra simples

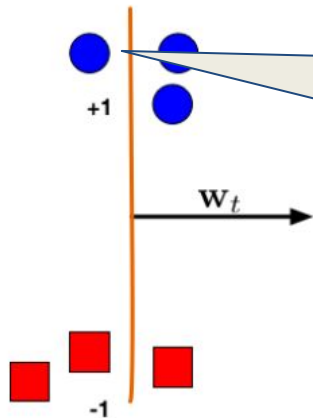
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture5>



- Não está classificado corretamente
 $y_0 = -1$
 $y = 1$
- net precisa aumentar
- w precisa aumentar
- somar x a w

Lecture 5
"Perceptron"
Cornell CS4780

inberger

até 24:30

[www.youtube.c](https://www.youtube.com/watch?v=w17gVvI)

[ach?v=w17gVvI](https://www.youtube.com/watch?v=w17gVvI)

edY

Perceptron: regra simples

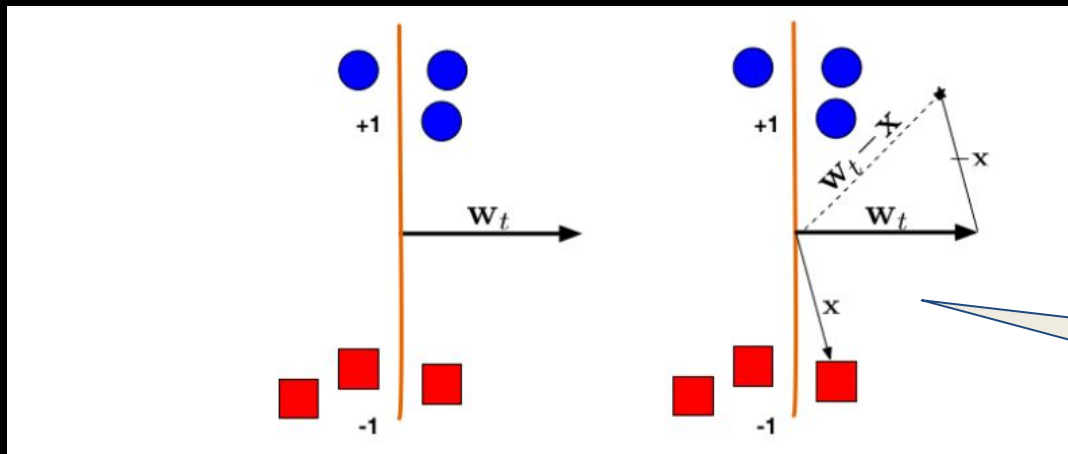
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

youtube.c

VvI

- Fizemos a suposição que $\eta=1$ no exemplo
- Se $\eta=0.01$ o que estamos fazendo?

Perceptron: regra simples

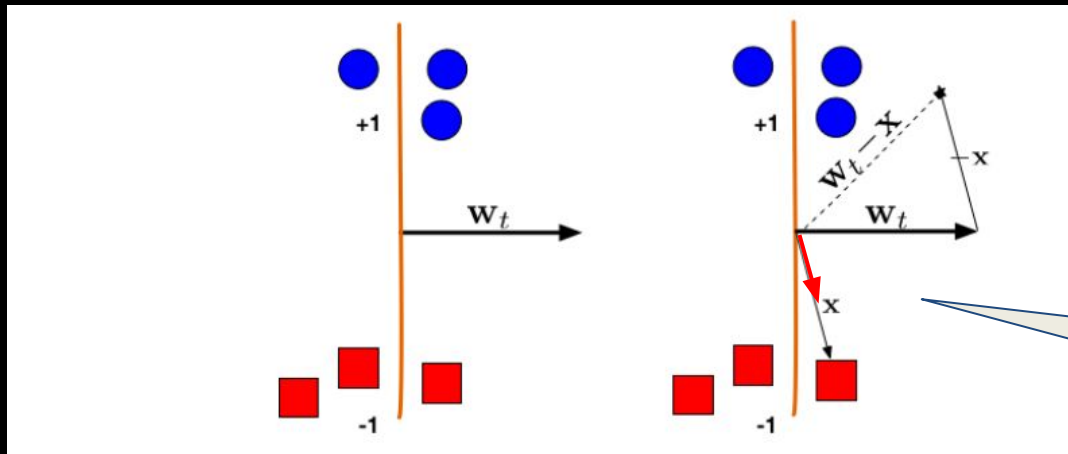
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

utube.c

VvI

- Fizemos a suposição que $\eta=1$ no exemplo
- Se $\eta=0.01$ o que estamos fazendo?

Perceptron: regra simples

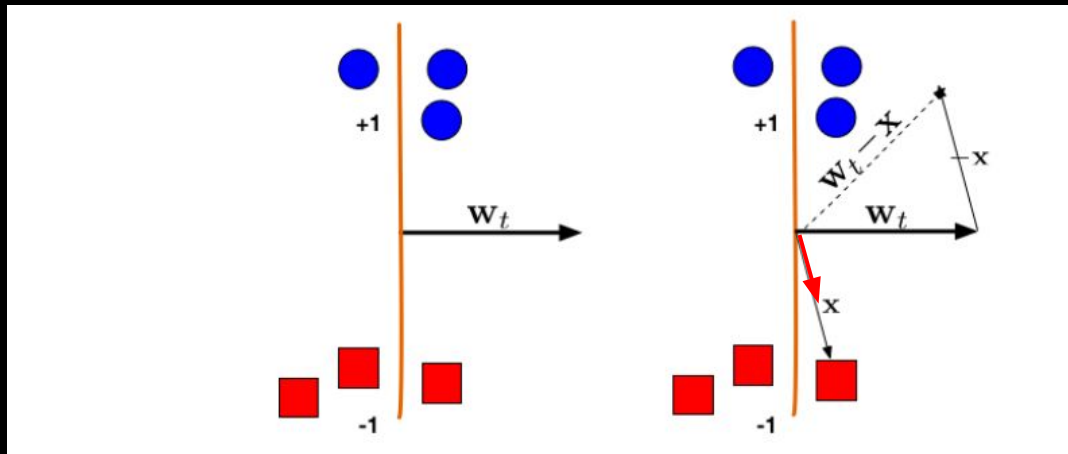
$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

$$y_0 = f(net)$$

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>



Lecture 5

"Perceptron"

-Cornell CS4780

SP17

Kilian Weinberger

minuto 22 até 24:30

<https://www.youtube.com/watch?v=w17gVvI-HuY>

Perceptron: regra simples

- Se $y_0 \neq y$, o vetor de pesos conectando as entradas a saída e o bias do neurônio são atualizados de acordo com as seguintes regras:

$$w_i(t + 1) = w_i(t) + \eta * y * x_i$$

$$\theta(t + 1) = \theta(t) + \eta * y$$

$$net = \sum_{i=1}^n x_i w_i + \theta$$

Foi demonstrado que se um conjunto de dados for linearmente separável, o Perceptron encontrará um hiperplano separador em um número finito de atualizações. Se os dados não forem linearmente separáveis, ele ficará em loop infinito.

Perceptron: regra simples

Perceptron Algorithm

Nesse vetor também
está incluído o θ

```
Initialize  $\vec{w} = \vec{0}$  // Initialize  $\vec{w}$ .  $\vec{w} = \vec{0}$  misclassifies everything.
while TRUE do // Keep looping
     $m = 0$  // Count the number of misclassifications,  $m$ 
    for  $(x_i, y_i) \in D$  do // Loop over each (data, label) pair in the dataset,  $D$ 
        if  $y_i(\vec{w}^T \cdot \vec{x}_i) \leq 0$  then // If the pair  $(\vec{x}_i, y_i)$  is misclassified
             $\vec{w} \leftarrow \vec{w} + y\vec{x}$  // Update the weight vector  $\vec{w}$ 
             $m \leftarrow m + 1$  // Counter the number of misclassification
        end if
    end for
    if  $m = 0$  then // If the most recent  $\vec{w}$  gave 0 misclassifications
        break // Break out of the while-loop
    end if
end while // Otherwise, keep looping!
```

Perceptron: regra simples

Perceptron Algorithm

Initialize $\vec{w} = \vec{0}$

while TRUE **do**

$m = 0$

for $(x_i, y_i) \in D$ **do**

if $y_i(\vec{w}^T \cdot \vec{x}_i) \leq 0$ **then**

$\vec{w} \leftarrow \vec{w} + y\vec{x}$

$m \leftarrow m + 1$

end if

end for

if $m = 0$ **then**

break

end if

end while

$$net = \sum_{i=1}^n x_i w_i + \theta$$

// Initialize $\vec{w}$. $\vec{w} = \vec{0}$ misclassifies everything.

// Keep looping

// Count the number of misclassifications, m

// Loop over each (data, label) pair in the dataset, D

// If the pair $(\vec{x}_i, y_i)$ is misclassified

// Update the weight vector $\vec{w}$

// Counter the number of misclassification

// If the most recent $\vec{w}$ gave 0 misclassifications

// Break out of the while-loop

// Otherwise, keep looping!

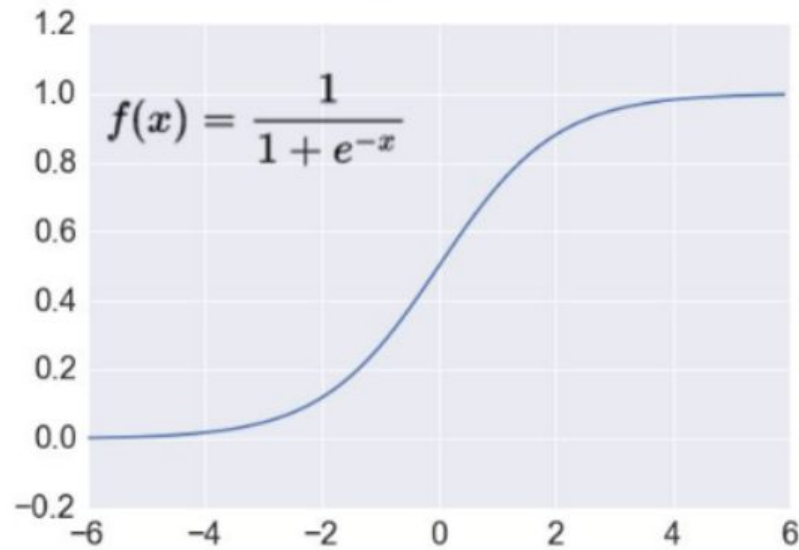
Observação sobre a função de ativação
usada pelo Perceptron

Perceptron:

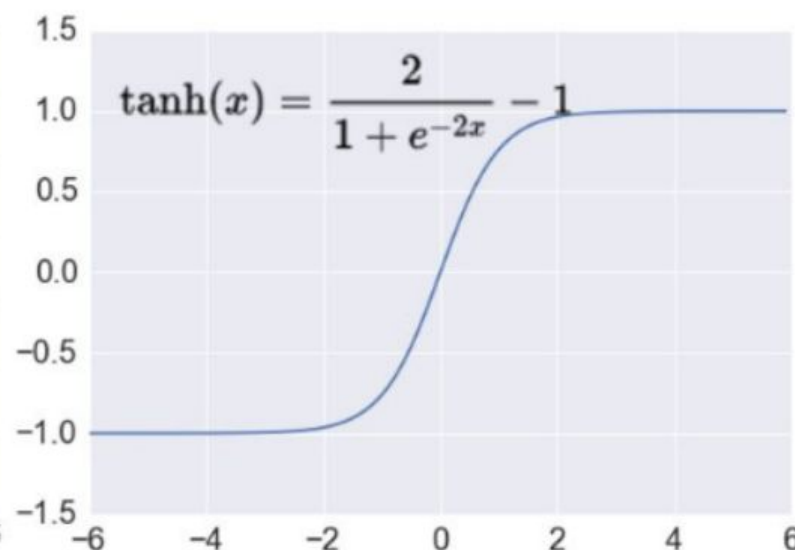
- Usa a função de ativação degrau

Outras funções de ativação

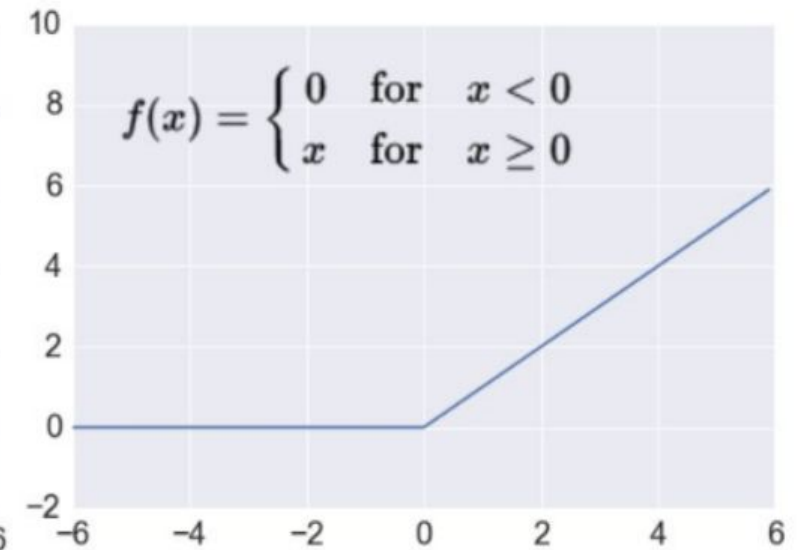
Sigmoid



TanH



ReLU



INTELIGÊNCIA ARTIFICIAL

Perceptron

Karina Valdivia Delgado